



GBase 8s 安全技术指南



GBase 8s 安全技术指南，南大通用数据技术股份有限公司

GBase 版权所有©2004-2030，保留所有权利

版权声明

本文档所涉及的软件著作权及其他知识产权已依法进行了相关注册、登记，由南大通用数据技术股份有限公司合法拥有，受《中华人民共和国著作权法》、《计算机软件保护条例》、《知识产权保护条例》和相关国际版权条约、法律、法规以及其它知识产权法律和条约的保护。未经授权许可，不得非法使用。

免责声明

本文档包含的南大通用数据技术股份有限公司的版权信息由南大通用数据技术股份有限公司合法拥有，受法律的保护，南大通用数据技术股份有限公司对本文档可能涉及到的非南大通用数据技术股份有限公司的信息不承担任何责任。在法律允许的范围内，您可以查阅，并仅能够在《中华人民共和国著作权法》规定的合法范围内复制和打印本文档。任何单位和个人未经南大通用数据技术股份有限公司书面授权许可，不得使用、修改、再发布本文档的任何部分和内容，否则将视为侵权，南大通用数据技术股份有限公司具有依法追究其责任的权利。

本文档中包含的信息如有更新，恕不另行通知。您对本文档的任何问题，可直接向南大通用数据技术股份有限公司告知或查询。

通讯方式

南大通用数据技术股份有限公司

天津市高新区开华道22号普天创新产业园东塔20-23层

电话：400-013-9696

邮箱：info@gbase.cn

商标声明

GBASE[®] 是南大通用数据技术股份有限公司向中华人民共和国国家商标局申请注册的注册商标，注册商标专用权由南大通用数据技术股份有限公司合法拥有，受法律保护。未经南大通用数据技术股份有限公司书面许可，任何单位及个人不得以任何方式或理由对该商标的任何部分进行使用、复制、修改、传播、抄录或与其它产品捆绑使用销售。凡侵犯南大通用数据技术股份有限公司商标权的，南大通用数据技术股份有限公司将依法追究其法律责任。

目 录

1 简介.....	1
2 管理员分类与职责.....	2
2.1 管理员的分类.....	2
2.2 数据库登录模式分类.....	2
2.2.1 功能概述.....	2
2.2.2 配置说明.....	3
3 身份鉴别.....	4
3.1 功能概述.....	4
3.1.1 用户标识.....	4
3.1.2 身份鉴别.....	4
3.2 配置说明.....	4
3.2.1 强化管理的口令鉴别.....	4
3.2.2 密码复杂度检查.....	5
3.2.3 密码时效性检查.....	5
3.2.4 尝试次数限制.....	5
3.2.5 用户密码传输加密.....	5
3.2.6 超时认证.....	6
3.2.7 支持证书鉴别.....	6
4 自主访问控制(Discretionary access control).....	7
4.1 功能概述.....	7
4.2 基本语法.....	7
4.3 权限划分.....	10
4.4 数据库级别权限.....	10
4.5 表和字段级权限.....	13
4.6 分片表（Fragment）权限.....	17
4.7 视图权限.....	17
4.8 SPL(存储过程、函数、触发器)的权限.....	18
4.9 用户自定义类型 UDT 权限.....	19
4.10 序列（Sequence）、同义词（Synonym）级别权限.....	20
5 标记与强制访问控制.....	21
5.1 功能概述.....	21
5.2 配置说明.....	21
6 数据保护.....	22

6.1 数据存储保密性.....	22
6.1.1 功能概述.....	22
6.1.2 配置说明.....	22
6.2 数据传输保密性.....	22
6.2.1 功能概述.....	22
6.2.2 配置说明.....	22
6.2.3 重启服务.....	23
7 备份与恢复.....	24
7.1 功能概述.....	24
7.2 使用.....	24
8 资源管理.....	25
8.1 功能概述.....	25
8.2 会话限制.....	25
8.3 低限报警.....	25
8.4 资源占用监控.....	26
9 安全审计.....	27
9.1 功能概述.....	27
9.2 审计事件.....	27
9.2.1 审计事件.....	27
9.2.2 配置说明.....	28
9.3 审计日志.....	28
9.3.1 审计日志配置.....	28
9.3.2 审计日志一键迁出.....	28
9.4 审计关联分析.....	30
9.4.1 异常检测.....	30
9.4.2 侵害分析.....	30
10 安全总开关及安全配置项汇总.....	30
10.1 功能概述.....	30
10.2 配置说明.....	30
11 使用示例.....	34
11.1 通用功能.....	34
11.1.1 数据库安装实例路径.....	34
11.1.2 获取环境变量.....	34
11.1.3 启动实例.....	34

11.1.4 停止实例.....	34
11.1.5 企业管理器连接数据库实例（sysdba/GBase8szyj）.....	34
11.1.6 安全管理器连接数据库实例（sysdsa/GBase8szyj）.....	34
11.1.7 安全管理器连接数据库实例（sysdaa/GBase8szyj）.....	35
11.1.8 创建数据库.....	35
11.1.9 删除数据库.....	35
11.1.10 创建表.....	35
11.1.11 修改表名.....	36
11.1.12 删除表.....	36
11.1.13 创建视图.....	36
11.1.14 删除视图.....	36
11.1.15 查询表结构.....	36
11.1.16 插入数据.....	37
11.1.17 修改数据.....	37
11.1.18 查询数据.....	37
11.1.19 删除数据.....	38
11.1.20 修改表.....	38
11.1.21 自主访问控制权限的赋予与回收.....	38
11.1.22 权限传播机制.....	39
11.1.23 标记功能.....	39
11.1.24 标记的粒度.....	40
11.1.25 标记的输入与输出.....	41
11.1.26 备份与恢复.....	42
11.1.27 客体重用.....	43
11.1.28 会话机制（最大连接数）.....	44
11.1.29 低限报警.....	44
11.1.30 资源占用监控.....	45
11.1.31 导入文件类型筛选.....	45
11.2 数据库安全功能.....	45
11.2.1 管理员身份鉴别.....	45
11.2.2 口令要求.....	46
11.2.3 用户尝试次数限制功能.....	46
11.2.4 口令更换周期.....	47
11.2.5 超时认证.....	47

11.2.6 管理员权限划分.....	47
11.2.7 审计日志一键迁出.....	47
11.2.8 审计策略设置.....	49
11.2.9 关联分析.....	49
11.2.10 日志数据安全.....	49

1 简介

本手册介绍了 GBase 8s 数据库产品的安全技术。

GBase 8s 是一款安全数据库，因此安全技术是 GBase 8s 数据库产品的核心价值所在。本手册描述了 GBase 8s 数据库的管理员分类与职责，以及安全模块的全部功能，分别是身份鉴别、自主访问控制、标记与强制访问控制、数据保护、备份与恢复、资源管理、安全审计和安全总开关配置管理。对于这些功能，本手册详尽地介绍了其功能概述、配置说明、基本语法等信息。手册的最后提供了针对安全模块功能的使用示例，涵盖了通用功能和数据库安全功能，仅供用户参考。

本手册的读者包括并不局限于以下人员：

- 数据库管理员
- 数据库服务器管理员
- 操作系统管理员
- 数据库系统安全维护员

2 管理员分类与职责

2.1 管理员的分类

系统提供三个方面的管理员操作：数据管理员操作、安全管理员操作和审计管理员操作。为此，系统设置了三类管理人员：数据管理员、安全管理员和审计管理员。

数据管理员使用企业管理器负责数据的管理、用户管理以及设定用户对数据库对象（库、表、字段等）的操作权限（DDL、DML 等）。在系统初始版本生成时建立并激活第一个负责数据管理的数据管理员 sysdba 并定义其初始口令，为 GBase8szyj。

安全管理员使用安全管理器进行系统的强制访问控制管理。包括安全标记设置、授权及安全审计员日志的审查分析，它为每个主体和每个客体设置安全级别，包括密级、范畴和访问特性。任何经过该软件的操作都要由统一的安全机制对主体及客体的安全级别进行比较审查。强制访问控制安全管理员还可以根据需要，修改自己的口令。具体地讲，安全管理员负责所有强制访问控制命令的操作。其具体职能是：

- 标识、修改（安全标签）
- 标识、修改与删除客体标识
- 标识、修改与删除主体标识
- 有关安全信息的浏览

在系统初始版本生成时建立并激活数据安全管理员 sysdsa 并定义其初始口令，为 GBase8szyj。

审计管理员使用审计工具进行审计功能的设置、维护和管理操作，负责系统审计功能的管理。对用户 in 数据库中的活动事件进行记录，该活动可能是修改或展示数据，或者是审计和安全策略的配置等。在系统初始版本生成时建立并激活审计管理员 sysdaa 并定义其初始口令，为 GBase8szyj。

2.2 数据库登录模式分类

2.2.1 功能概述

GBase 8s Server 提供三种用户登录模式，分别是数据库内部用户模式、维护模式和全模式，描述如下：

- **数据库内部用户模式：**仅支持 sysdba、sysdsa、sysdaa 以及由 sysdba 创建的用户使用数据库。
- **维护模式：**仅支持 root、gbasedbt 用户访问数据库，可用于进行数据库备份恢复、导入导出等工作。该模式下不支持数据库内部用户登录。

- **全模式：**同时支持数据库内部用户模式和维护模式。

2.2.2 配置说明

用户登录模式功能可通过设置 ONCONFIG 文件 CONF_SWITCH 配置项控制登录模式。推荐使用CONF_SWITCH=2 模式，可通过 onmodeadmin -wf CONF_SWITCH=2 设置。

CONF_SWITCH 值	行为说明
0	1) 仅 sysdba、sysdsa、sysdaa 以及由 sysdba 创建的用户可使用数据库 2) root、gbasedbt 用户不可登录数据库 3) onbar, ontape, onparams, onspaces, dbexport, dbimport, archecker, onunload, onload 不可用
1	1) 仅 root、gbasedbt 可登录数据库，其余用户不可登录数据库 2) onbar, ontape, onparams, onspaces, dbexport, dbimport, archecker, onunload, onload 可用
2	同时支持 CONF_SWITCH 值为 0 和 1 的行为。

3 身份鉴别

3.1 功能概述

DBMS 的身份鉴别功能分为两个部分：用户标识与身份鉴别。

3.1.1 用户标识

每个进入 DBMS 的用户首先需要有一个用户标识，并在 DBMS 的整个生命周期实现用户标识的唯一性。

3.1.2 身份鉴别

系统按照基本鉴别、不可伪造鉴别及一次性使用鉴别要求进行用户身份鉴别，用户在使用 DBMS 时必须首先给出用户标识，通过检验合格后才能进入使用 DBMS。

身份鉴别功能实现包括：

- **强化管理的口令鉴别：**系统采用存储哈希口令的方式提高口令安全性。
- **密码复杂度检查：**口令长度不得低于 8 位，口令必须包含大写字母、小写字母、数字。
- **密码时效性检查：**用户口令有效期最多为 7 天，超过 7 天未修改口令，登录时需要重新设置口令。
- **尝试次数限制：**提供用户鉴别尝试次数限制功能，用户可设定尝试次数，最多为 5 次。超过后用户挂起，间隔一段时间后可以正常登录。
- **用户密码传输加密：**用户在客户端登录数据库服务器时，输入的密码信息进行加密传输，server 端进行密码解密并校验。
- **超时认证：**用户登录数据库长时间空闲不进行操作，再次操作时，需要输入管理员信息重新登录。
- **支持证书鉴别：**采用 SSL 协议，通过双向认证，即客户端或工具端在申请访问服务端时，利用对方的验证算法来验证通信中收到的签名信息。

3.2 配置说明

3.2.1 强化管理的口令鉴别

该功能无需配置，GBase 8s 对用户密码采用 sha-256 和 Base64 编码方式进行加密存储。

3.2.2 密码复杂度检查

ONCONFIG 文件中 PWDCOMPLEXCHK 配置项设置为 1 时，启动密码复杂度检查模块，设置为 0 时，关闭此模块。

ONCONFIG 文件中 PWD_MIN_LENGTH 配置项设置密码最小长度，最小值为 8，即密码长度不低于 8 位，最大值 32。启用密码复杂度检查模块后，该参数生效。

更改 PWDCOMPLEXCHK 和 PWD_MIN_LENGTH 后，需要重启数据库才可生效。

【示例 11.2.2】

3.2.3 密码时效性检查

ONCONFIG 文件中 PWDDATECHK 配置项设置为 1 时，启动密码时效性检查模块，设置为 0 时，关闭此模块。

ONCONFIG 文件中 CFPWDPERIOD 配置项设置最长更换周期（天），最大值为 7。启用本模块后，该参数生效。

更改 PWDDATECHK 和 CFPWDPERIOD 后，需要重启数据库才可生效。

密码使用时间超出最长更换周期后，用户可通过企业管理器、安全管理器、审计工具登录数据库，在登录后的弹出窗体中更新密码。

【示例 11.2.4】

3.2.4 尝试次数限制

ONCONFIG 文件中 CF_LIMIT_SWITCH 配置项设置为1时，启动尝试次数限制模块，设置为0时，关闭此模块。

ONCONFIG 文件中 CFLIMITNUM 配置项设置登录失败超次阈值，单位为天，最大值 7。启用本模块后，该参数生效。

ONCONFIG 文件中 LOCKPERIOD 配置项设置登录失败超过阈值后，用户被锁定时间，单位为天，最大值为 7。启用本模块后，该参数生效。

更改 CF_LIMIT_SWITCH、CFLIMITNUM 和 LOCKPERIOD 后，需要重启数据库才可生效。

【示例 11.2.3】

3.2.5 用户密码传输加密

该功能无需配置，GBase 8s 对用户密码采用 AES 算法进行加密。

3.2.6 超时认证

ONCONFIG 文件中 SESTO_SWITCH 配置项设置为 1 时,启动超时认证模块,设置 0 时,关闭此模块。

ONCONFIG 文件中 SES_TIMEOUT 配置项设置超时时间,单位为秒,最小值为 1,最大值 2147483647 (约 68 年)。如设置 SES_TIMEOUT 为 600,表示用户登录数据库后,超过 600 秒进行操作,再次操作时,需要重新登录。启用本模块后,该参数生效。

更改 SESTO_SWITCH 和 SES_TIMEOUT 后,需要重启数据库才可生效。

【示例 11.2.5】

3.2.7 支持证书鉴别

GBase 8s 支持数字证书模式进行用户登录鉴别,配置方式见 6.2 数据传输保密性。

4 自主访问控制(Discretionary access control)

4.1 功能概述

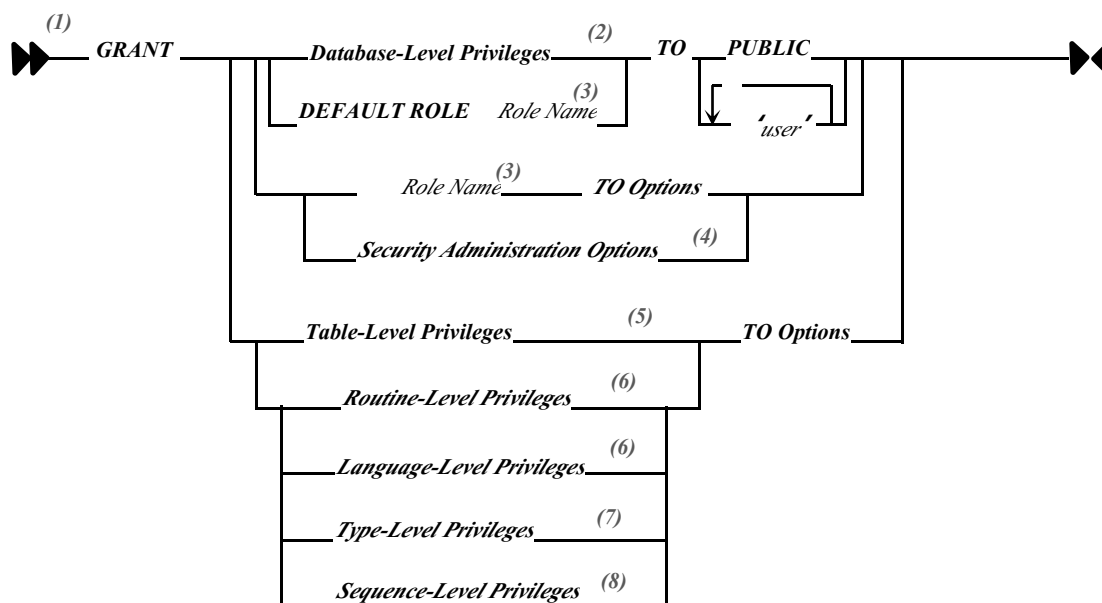
GBase 8s 提供的自主访问控制（Discretionary Access Control，DAC）功能非常灵活，用户可对不同的数据对象设定不同的访问权限，同时，不同的用户对同一数据对象也有不同的权限。用户可以将其拥有的访问权限转授给其他用户。自主访问控制是基于用户的，由数据对象的拥有者决定哪些用户可以对该数据对象进行访问，访问权限完全由数据对象的拥有者决定。

用户对数据对象的操作权利被称为权限，用户权限是由数据库对象（库、表、字段等）和操作类型（DDL、DML 等）两个要素组成，对用户权限的定义就是定义该用户可以对哪些数据库对象进行哪些类型的操作。

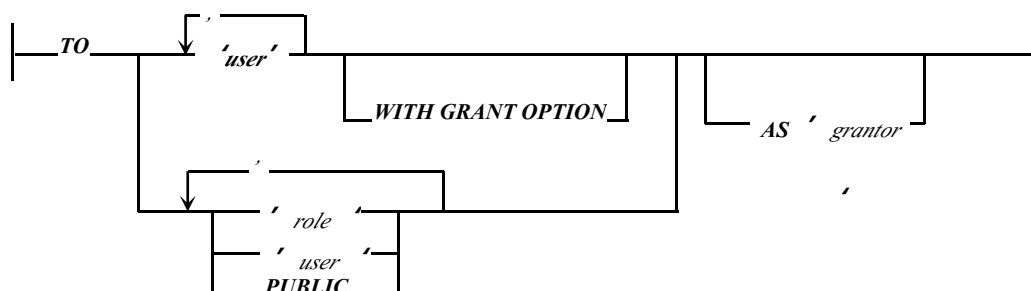
该功能不受配置参数控制。

4.2 基本语法

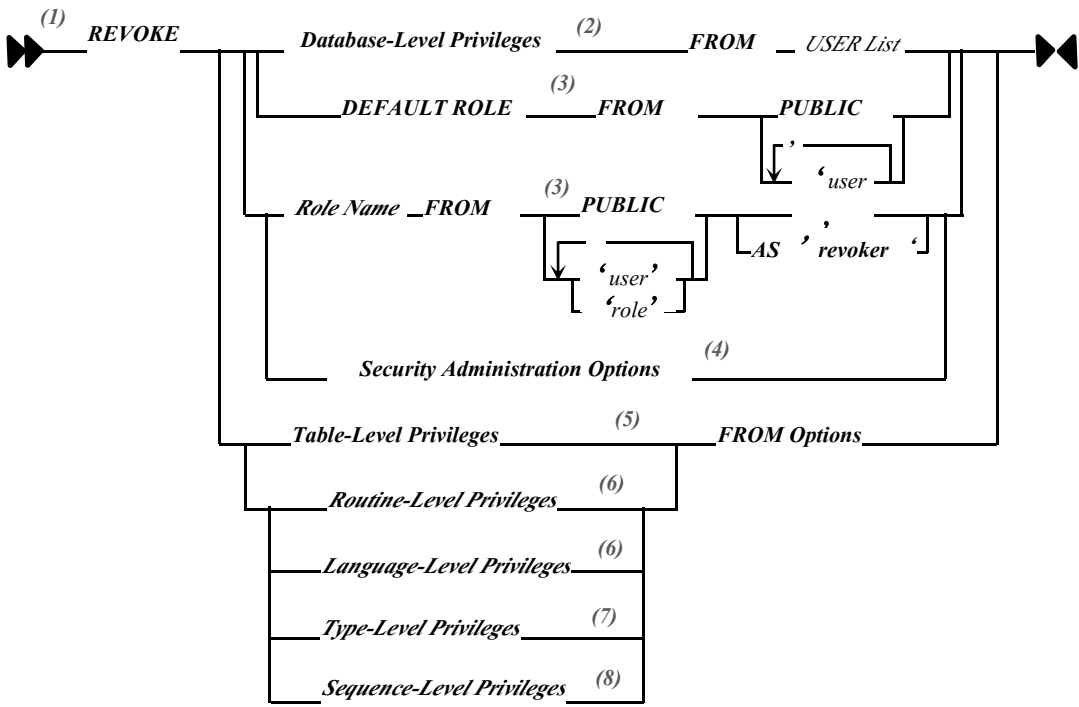
GBase 8s 的权限赋予语法如下：



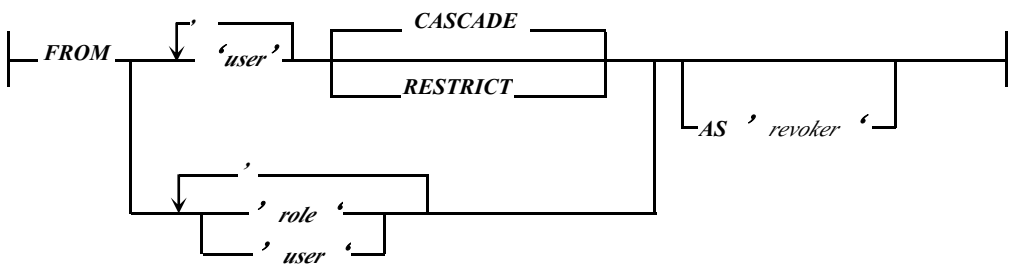
TO Options 子句语法如下：



收回权限的语法如下：



FROM Options 子句语法如下：



说明如下：

- 1) GBase 8s 赋权和收回权限关键字；
- 2) 数据库级别权限授权和收回；
- 3) 角色及默认角色授予和收回；
- 4) 基于标签的访问控制（LBAC）相关设置；
- 5) 表级（字段级）权限授权的授予和收回；
- 6) SPL（存储过程、函数、触发器）授权和收回；
- 7) 自定义数据类型授权的授权和收回；
- 8) 序列（Sequence）的授权和收回。

4.3 权限划分

在 GBase 8s 中，可以对数据库、表、字段、视图、SPL（存储过程、函数、触发器）、分片等数据库对象进行权限的授予和收回操作，如下图所示：

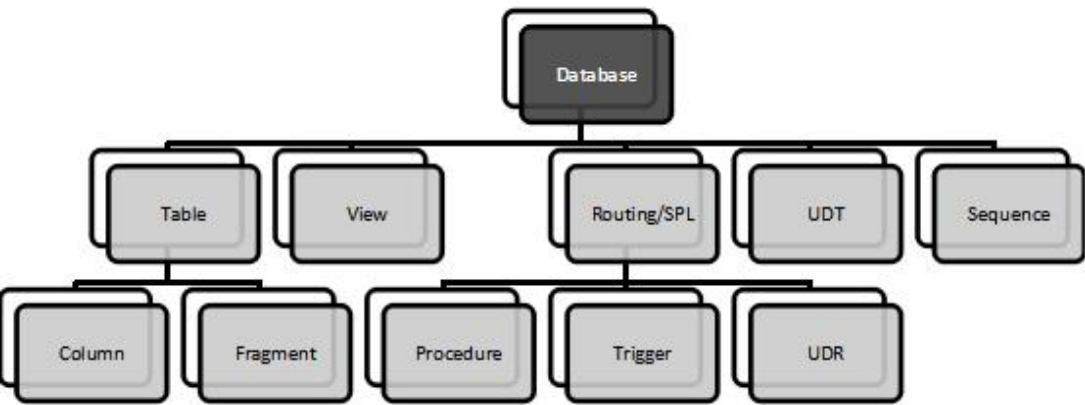


图 1 GBase 8s 安全数据库的不同安全层面

其中，数据库的权限是其他权限的基础，用户只有拥有数据库的权限后才能访问表、视图、存储过程等其他数据库资源。

数据库以及其中的每个表、视图、存储过程和自定义数据类型都具有所有者。对象的所有者通常是创建该对象的人员。数据库对象的所有者默认对该对象具有所有权限。

数据库不同元素的权限如下表所示：

对象	权限
Database	Connect, resource, DBA
Table	Select, update, insert, delete, index, alter, references
Column	Select, update, references
Fragment	Insert, update, delete
View	Select, insert, delete, update
Sequence	Select, alter
UDT	Usage, under
SPL	Execute

4.4 数据库级别权限

对于所有有权使用指定数据库的用户都必须赋予其数据库级别的用户权限。在 GBase 8s 中，数据库级别的用户权限有三种，按权限从低到高排列依次为：CONNECT、RESOURCE、DBA。

1. CONNECT

这是级别最低的一种数据库级别用户权限。拥有该权限的用户可以执行 SELECT、INSERT、UPDATE 和 DELETE 语句，针对数据表执行存储过程，创建数据表的视图，创建临时表。

具有 connect 特权的任何用户都可以执行以下的功能：

- 使用语句连接数据库
- 执行 Select, Insert, Update, Delete 语句（如果用户有必要的表级别权限）
- 创建视图，如果用户对基本表有 select 权限
- 创建同义词
- 创建临时表以及创建临时表的索引
- 改变或删除表或索引，如果用户拥有表或者索引
- 对表或者视图授予特权，如果用户拥有表

2. RESOURCE

拥有该权限的用户除拥有 CONNECT 的全部权限外，还可以创建新表，并可对自己创建的表执行 ALTER 和 DROP 操作，并执行创建索引，执行自定义函数（UDR）等操作。

除了具有 connect 特权的功能，Resource 特权的持有者可执行以下的功能：

- 创建新表
- 创建新索引
- 创建新 UDR
- 创建新数据类型

3. DBA

数据库的建立者和拥有者被自动受于这种权限。拥有该权限的用户除拥有 CONNECT、RESOURCE 的全部权限外，还可以对其他用户授予或解除 CONNECT、RESOURCE 和 DBA 权限，可以对其他用户创建的表和视图执行 DROP 和 ALTER 操作。也可以对数据库执行 DROP、START、STOP 和 RECOVER 操作。

拥有 Resource 特权的所有能力，并且能执行以下的附加操作：

- 将任何数据库特权（包括 DBA 特权）授予其他用户

- 将任何表级别特权授予其他用户和角色
- 创建表，视图和索引，指定其他用户作为这些对象的所有者
- 改变，删除或者重命名数据库对象，不管谁拥有它们
- 插入，删除或更新除 systables 之外的任何系统目录表的行

表 1 数据库级别权限表

权限	Connect	Resource	DBA
Select, insert, update, delete, use temporary tables, and use views.	Yes	Yes	Yes
Create, alter, drop, and index own tables.	No	Yes	Yes
Grant, revoke, drop other owned tables, and start and stop server.	No	No	Yes

■ 数据库权限的授予和撤销

在创建一个新的数据库后，创建者将对新创建的数据库拥有 DBA 权限。该用户连接到这个库后，通过 *GRANT* 命令可以对其他用户授予相应的数据库级别权限。

GBase 8s 使用 *PUBLIC* 关键字代表访问该数据库的所有用户。任何赋予 *PUBLIC* 的权限都会被当做用户的缺省权限。如果需要指定某个用户，使用该用户的用户 IDs。如需指定多个用户，多个用户名之间用逗号分隔。

- 赋予 *CONNECT* 权限，使用如下语法：

```
GRANT CONNECT TO PUBLIC;
GRANT CONNECT TO user1;
GRANT CONNECT TO usera, userb, userc;
```

- 收回 *CONNECT* 权限，使用如下语法：

```
REVOKE CONNECT FROM PUBLIC;
REVOKE CONNECT FROM user1;
REVOKE CONNECT FROM usera, userb, userc;
```

- 赋予 *RESOURCE* 权限，使用如下语法：

```
GRANT RESOURCE TO PUBLIC;  
GRANT RESOURCE TO user1;  
GRANT RESOURCE TO usera, userb, userc;
```

- 收回 **RESOURCE** 权限，使用如下语法：

```
REVOKE RESOURCE FROM PUBLIC;  
REVOKE RESOURCE FROM user1;  
REVOKE RESOURCE FROM usera, userb, userc;
```

- 赋予 **DBA** 权限，使用如下语法：

```
GRANT DBA TO user1;  
GRANT DBA TO usera, userb, userc;
```

- 收回 **DBA** 权限，使用如下语法：

```
REVOKE DBA FROM user1;  
REVOKE DBA FROM usera, userb, userc;
```

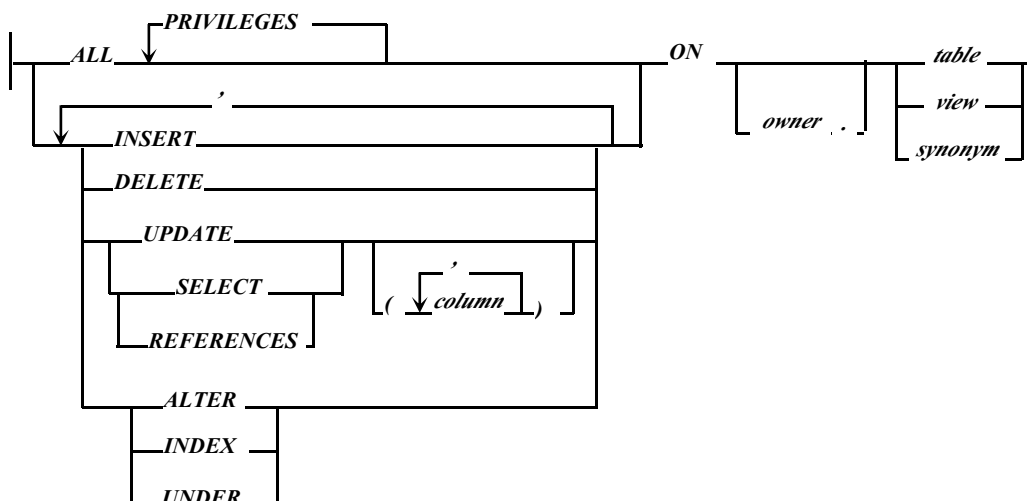
对于拥有 DBA、RESOURCE 权限的用户，收回其对应的 DBA、RESOURCE 权限，将自动将其权限降为 CONNECT 权限。而对于只拥有 CONNECT 权限的用户，收回其 CONNECT 权限，实际上也就意味着删除该数据库用户。

4.5 表和字段级权限

当一个用户访问数据库时，该数据库的 DBA 可以限制其对数据库内指定表和表中指定字段的访问。任何 RESOURCE 级别或 DBA 级别的用户都可以创建表。表的拥有者或任何 DBA 级别的用户都可以对其他用户授予或收回针对该数据表的表级别和字段级别的权限。针对表和字段的权限共有 8 种：INSERT、DELETE、SELECT、UPDATE、REFERENCES、INDEX、ALTER、ALL。

与数据库级别的权限相同，用户可以采用 GRANT 和 REVOKE 命令对其他用户授予和收回对用户表级别和字段级别的权限。

基本语法如下：



对于表和字段权限的授予和收回示例如下：

1. INSERT

该权限表示用户可以向表中增添新的数据。

授权语法示例如下：

```

GRANT INSERT ON customer_table TO user1;
GRANT INSERT ON customer_table TO usera, userb, userc;
GRANT INSERT ON customer_table TO PUBLIC;
  
```

收回权限语法示例如下：

```

REVOKE INSERT ON customer_table FROM PUBLIC;
REVOKE INSERT ON customer_table FROM usera, userb, userc;
REVOKE INSERT ON customer_table FROM user1;
  
```

2. DELETE

该权限表示用户可以删除表中的数据。

授权语法示例如下：

```
GRANT DELETE ON customer_table TO user1;  
GRANT DELETE ON customer_table TO usera, userb, userc;  
GRANT DELETE ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE DELETE ON customer_table FROM PUBLIC;  
REVOKE DELETE ON customer_table FROM usera, userb, userc;  
REVOKE DELETE ON customer_table FROM user1;
```

3. SELECT

该权限可以扩展到指定字段，表示用户可以查询表中的记录或指定字段，授权语法示例如下：

```
GRANT SELECT ON customer_table TO usera, userb, userc;  
GRANT SELECT (customer_id, customer_name) ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE SELECT (customer_id, customer_name) ON customer_table FROM PUBLIC;  
REVOKE SELECT ON customer_table FROM usera, userb, userc;
```

4. UPDATE

该权限可以扩展到指定字段，表示用户可以修改表中的记录或指定字段，授权语法示例如下：

```
GRANT UPDATE ON customer_table TO usera, userb, userc;  
GRANT UPDATE (customer_id, customer_name) ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE UPDATE (customer_id, customer_name) ON customer_table FROM PUBLIC;  
REVOKE UPDATE ON customer_table FROM usera, userb, userc;
```

5. REFERENCES

该权限可以扩展到指定字段，表示用户可以针对表中的记录、指定字段制定参考性约束条件。参考性约束条件用来执行诸如多级删除和字段之间关联的工作。授予该权限的用户至少是 RESOUCE 级别的用户，授权语法示例如下：

```
GRANT REFERENCES ON customer_table TO usera, userb, userc;  
GRANT REFERENCES (customer_id, customer_name) ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE REFERENCES (customer_id, customer_name) ON customer_table FROM PUBLIC;  
REVOKE REFERENCES ON customer_table FROM usera, userb, userc;
```

6. INDEX

该权限表示用户可以建立或删除与指定表有关的索引。授予该权限的用户应该至少是 RESOURCE 级别的用户。一个 CONNECT 级别的用户即使被授予该权限也不能建立永久索引，但具有 connect 权限的用户可以创建临时表的索引。该权限不能扩展到指定字段，因为索引是建立在表的所有行的基础上，授权语法示例如下：

```
GRANT INDEX ON customer_table TO usera, userb, userc;  
GRANT INDEX ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE INDEX ON customer_table FROM PUBLIC;  
REVOKE INDEX ON customer_table FROM usera, userb, userc;
```

7. ALTER

该权限表示用户可以改变表的结构。所谓改变表的结构是指添加或者删除行，修改列数据类型，添加或者删除约束，将表的锁定方式从 page 更改为 row，或为表添加或删除相应的行类型名称。该权限只能针对数据表。授予该权限的用户必须是DBA级别的用户。授权语法示例如下：

```
GRANT ALTER ON customer_table TO usera, userb, userc;  
GRANT ALTER ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE ALTER ON customer_table FROM PUBLIC;  
REVOKE ALTER ON customer_table FROM usera, userb, userc;
```

8. ALL

该权限包含上面的所有 7 种权限。授权语法示例如下：

```
GRANT ALL ON customer_table TO usera, userb, userc;  
GRANT ALL ON customer_table TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE ALL ON customer_table FROM PUBLIC;  
REVOKE ALL ON customer_table FROM usera, userb, userc;
```

● 权限组合

可以使用一条命令同时针对某一（或多个）用户授予或解除多种表级别或字段级别的权限。例如同时授予所有用户对表 *customer_table* 的 *INSERT*、*DELETE* 和 *UPDATE* 权限：

```
GRANT INSERT, DELETE, UPDATE ON customer_table TO PUBLIC;
```

又如使用一条命令同时收回用户 *user1* 对表 *customer_table* 的 *INDEX*、*SELECT* 权限以及对字段 *phone* 的 *ALTER* 权限：

```
REVOKE INDEX, SELECT, ALTER (phone) ON customer_table FROM user1;
```

【示例 11.1.21】

4.6 分片表（Fragment）权限

分片（Fragment）是 GBase 8s 数据库服务器提供的一种内部的把大表拆分为多个小表，同时对外提供同一个表的管理方式。有了 *fragment* 层面的权限后，可以对“分片的数据表”实现更细粒度的权限控制。

例如，下面的示例表示仅赋予用户 *user1* 对数据表某个分片的访问权限：

```
REVOKE FRAGMENT ALL ON customer_table FROM USER1;  
GRANT FRAGMENT INSERT, UPDATE, DELETE ON customer_table (dbspace1) TO user1;
```

4.7 视图权限

视图是由存储在数据库中的查询定义的虚拟表（Virtual Table）。视图所对应的查询被称为视图定义，它规定了如何从一个或者几个基本表（Base Table）中导出视图。在数据库中只存放视图的定义，不存放视图对应的数据，数据仍存放在导出视图的基本表中。修改基本表中的数据，相应视图中的数据也随之改变。视图就像一个窗口，透过视图可以看到数据库中待观察的数据。

在创建视图时，创建者需要对数据库拥有 *Resource* 或以上权限。

由于视图是不实际存储数据的虚拟表，因此对视图的更新最终要转换为对基本表的更新。同理，对于视图的访问，权限划分为对视图和导出视图的基本表两部分，对于视图的访问，权限要求视图使用者同时拥有视图和导出视图的基本表的权限。其权限

判定规则如下表所示：

表 1 视图权限判定

视图权限	创建者 基本表权限	访问者 基本表权限	权限判定
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	Yes	Yes
Yes	No	No	No
No	Yes	Yes	No
No	Yes	No	No
No	No	Yes	No
No	No	No	No

对于视图权限的授予和收回，语法等同于对表的权限授予和收回，示例如下：

创建视图示例：

```
GRANT VIEW customer_view AS
SELECT *
FROM customer_table
WHERE AS customer_id = 199;
```

授权语法示例如下：

```
GRANT SELECT, UPDATE ON customer_view TO usera, userb, userc;
GRANT ALL ON customer_view TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE ALL ON customer_view FROM PUBLIC;
REVOKE UPDATE ON customer_view FROM usera, userb, userc;
```

4.8 SPL(存储过程、函数、触发器)的权限

SPL（Stored Procedure Language）是在 GBase 8s 中提供的一种流程控制（分支和循环）的 SQL，是 GBase 8s 对标准SQL的扩充，用于编写存储过程。GBase 8s 的存储过程包括 SPL 过程（没有返回值）和 SPL 函数（有返回值）。在创建 SPL 时，创建者需要对数据库拥有 Resource 或以上权限。

在 GBase 8s 中，触发器（Trigger）是一个特殊的存储过程。在对表执行 INSERT、DELETE 或 UPDATE 操作时，通过定义触发器使 GBase 8s 自动调用存储过程或SQL语句。由于触发器是由系统所调用执行，所以触发器不涉及执行权限。

同视图相似，对于 SPL 执行时的权限判定也受SPL本身权限和 SPL 内SQL 访问的基本表权限所限制。在 GBase 8s 的权限判定规则中，只要 SPL 的创建者和使用者其

中有一人对被 SPL 访问的基本表有权限，即可判定该 SPL 权限通过，其判定规则如下表所示：

表 1 SPL权限判定

SPL权限	创建者 基本表权限	访问者 基本表权限	权限判定
Yes	Yes	Yes	Yes
Yes	Yes	No	Yes
Yes	No	Yes	Yes
Yes	No	No	No
No	Yes	Yes	No
No	Yes	No	No
No	No	Yes	No
No	No	No	No

对于存储过程权限的授予和收回，语法同于对表的权限授予和收回相似，示例如下：

授权语法示例如下：

```
GRANT EXECUTE ON square(x INT) TO usera, userb, userc ;           -- function
GRANT EXECUTE ON proc_abc TO PUBLIC;                             -- procedure
```

收回权限语法示例如下：

```
REVOKE EXECUTE ON proc_abc FROM PUBLIC;
REVOKE EXECUTE ON square(x INT) FROM usera, userb, userc;
```

4.9 用户自定义类型 UDT 权限

GBase 8s 支持用户自定义的数据类型(UDT：User Define Type)。创建用户自定义的数据类型时，只有 DBA 或该数据类型的拥有者才能够授予或撤销该自定义类型级别的权限(这些权限控制哪些用户可以使用该UDT)。针对 UDT，支持两种类型级别的权限控制。

对用户自定义的类型的 Usage 权限：当用户创建了自定义数据类型后，其他用户必须通过 grant 语句得到 usage 权限后才可以使用这些 UDT。

对该 UDT 的 Under 特权：用户 user1 拥有创建的已命名的 UDT ty_a。如果 user1 想要用户 user2 能够在 UDT a 上创建子类型 ty_b，则必须将 ty_a 的 under 特权授予用户 user2。

授权语法示例如下：

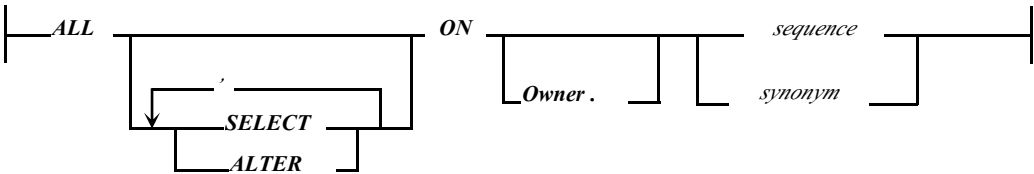
```
GRANT USAGE ON TYPE ty_a TO usera, userb, userc ;
GRANT UNDER ON TYPE ty_a TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE UNDER ON ty_a FROM PUBLIC;
REVOKE USAGE ON ty_a FROM usera, userb, userc;
```

4.10 序列（Sequence）、同义词（Synonym） 级别权限

GBase 8s 对于序列和同义词提供两种操作的权限管理，Select 和 Alter，其语法如下：



授权语法示例如下：

```
GRANT ALL ON sequence_1 TO usera, userb, userc ;
GRANT SELECT ON synonym_2 TO PUBLIC;
```

收回权限语法示例如下：

```
REVOKE SELECT ON synonym_2 FROM PUBLIC;
REVOKE ALL ON sequence_1 FROM usera, userb, userc;
```

【示例 11.1.21】

5 标记与强制访问控制

5.1 功能概述

DBMS 中的主体与客体均需标以敏感标记（简称标记），标记分为安全等级标记与范畴标记，等级标记是用正整数表示，而范畴标记则用集合表示。

由负责 MAC 管理的的安全管理员设定主体和客体的密级和范畴。

在标记的基础上可以进行强制访问控制，强制访问控制采用 Bell Lapadula 访问策略模型，即当主体访问客体时，对主体与客体的标记作比较，一般而言，只有当主体安全等级标记大于等于客体安全等级标记并且其主体范畴标记包含客体范畴标记时，才能进行读访问（称向下读原则）。只有当主体安全等级标记小于等于客体安全等级标记并且其主体范畴标记被客体范畴标记包含时，才能进行写（包括修改、删除）访问（称向上写原则）。Bell-La Padula 模型的基本安全策略是“下读上写”，在 GBase 8s 中，基于实用性的考虑对安全策略进行了优化。默认的安全访问策略为：下读等写。

当主体访问客体时，强制访问控制需按安全访问策略模型作安全检查，只有当符合模型要求时访问才能进行，否则为非法，访问不能进行。

强制访问控制粒度支持记录、字段级。

5.2 配置说明

该功能可通过设置 ONCONFIG 文件 LBAC_SWITCH 配置项控制是否启用。当 LBAC_SWITCH 设置为1时，启用此功能。LBAC_SWITCH 设置为 0 时，关闭此功能。更改LBAC_SWITCH后，需要重启数据库，方可生效。

另外，启用该功能后，用户才可以在安全管理器中使用该功能，详见《安全管理器使用手册》。

【示例 11.1.23】

【示例 11.1.24】

【示例 11.1.25】

6 数据保护

6.1 数据存储保密性

6.1.1 功能概述

存储数据的保密性是安全数据库的最重要的功能之一。GBase 8s 安全数据库的数据加密采用库内加密的方式，在数据库管理系统的内核存储引擎级进行数据加解密处理，即数据在进行物理 I/O 时完成加解密工作。

支持 onbar、ontape 在加密模式下进行备份恢复。

由于数据页只有在真正进行 I/O 时才进行加解密操作，从而对于合法用户来讲是完全透明的，因此也可以称为透明存储加密。

密/明文：数据库数据经加密后落地，即为密文。从磁盘加载后经数据库解密即为明文。

6.1.2 配置说明

ONCONFIG 文件 SERVER_ENCRYPTION_TYPE 配置项用来设定数据存储类型，支持明文、软加密和加密卡三种方式。其中 CLEARTEXT 为明文存储；SOFT 为软加密方式存储。如需支持其他加密方式，请联系 GBase 8s 技术支持人员获得更进一步的信息。

更改数据存储方式，即修改 SERVER_ENCRYPTION_TYPE 参数值，需要对数据库进行初始化，即执行 `oninit -ivy`，方可正常使用。初始化后，数据库将清除用户数据。

6.2 数据传输保密性

6.2.1 功能概述

用户端与数据库服务端的信息访问，使用以 SSL 为基础的安全协议来建立安全保密数据传输路径。使用加密算法保护链路层安全，使用证书对服务端和客户端进行双向验证，保证客户端和服务端之间通讯的保密性和完整性。

6.2.2 配置说明

GBase 8s Server 启用 SSL 模块，涉及三个配置文件，ONCONFIG、SQLHOSTS、SQLHOSTS.EXT 文件。

1. ONCONFIG 设置 VPCLASS、NETTYPE 参数，示例如下：

```
VPCLASS encrypt,num=1
NETTYPE onsocssl,1,150,CPU
```

2. SQLHOSTS 设置 onsocssl 通信类型，示例如下：

```
ol_gbase8s onsocssl 127.0.0.1 10603
```

3. SQLHOSTS.EXT 设置预置证书（DEMO）级，示例如下：

SQLHOSTS.EXT 文件为 /opt/gbase8s/etc/sqlhosts.ol_gbase8s.ext，初始化设置内容：

```
[config]
GBS_TYPE=gbasessl

[ol_gbase8s]
server=ol_gbase8s
client=ids1_client

[ol_gbase8s]
TLSCACertificateFile=/opt/gbase8s/ids_cfg/security/GBASECA.pem
TSLCertificateFile=/opt/gbase8s/ids_cfg/security/GBase8s.pem
TSLCertificateKeyFile=/opt/gbase8s/ids_cfg/security/GBase8s.key
TSLCertificateKeyFilePasswd=
TLSVerifyCert=verify

[ids1_client]
TLSCACertificateFile=/opt/gbase8s/ids_cfg/security/GBASECA.pem
TSLCertificateFile=/opt/gbase8s/ids_cfg/security/root.pem
TSLCertificateKeyFile=/opt/gbase8s/ids_cfg/security/root.key
```

GBase 8s 安装后，提供上述证书相关文件，供 demo 使用。如需使用正式证书文件，请联系 GBase 8s 技术支持人员获得更进一步的信息。

6.2.3 重启服务

服务端证书配置好后，需要重启 IDS 服务来生效。

7 备份与恢复

7.1 功能概述

GBase 8s 提供 onbar、ontape 备份恢复工具。他们都可以用来完成系统备份、逻辑日志备份以及系统恢复等功能，同时这两种工具也有着不同的特性。

7.2 使用

Gbackuprestore、ontape 的使用十分简单、方便，但必须在命令行环境中执行。Gbackuprestore 还支持将数据库实例恢复到某时间点的功能。同时支持在加密模式下进行数据备份与恢复，配置参数同数据存储保密。

onbar、ontape 用户同非安全版，具体使用说明详见《GBase 8s 备份与恢复指南》。

【示例 11.1.26】

8 资源管理

8.1 功能概述

资源管理主要包括对用户连接数量控制、服务器存储空间监控预警、数据空间占用监控等功能。

8.2 会话限制

GBase 8s 提供会话限制功能，即限制最大连接数（session）。

ONCONFIG 中 SES_NUM_SWITCH 配置项和 MAX_SESSION_NUM 配置项设置会话限制功能。

SES_NUM_SWITCH 配置项负责启停会话限制模块。当 SES_NUM_SWITCH 为 1 时，启动会话限制模块。当 SES_NUM_SWITCH 为 0，关闭该模块。该参数改变需要重启数据库才可生效。

SES_NUM_SWITCH 为 1 时，设置 MAX_SESSION_NUM 参数，确定最大用户连接数，如 MAX_SESSION_NUM 200，表示支持最大连接数为 200。该参数改变需要重启数据库才可生效。

8.3 低限报警

GBase 8s 提供服务器存储空间低限报警功能。用于监控根目录空间剩余值是否低于阈值，当磁盘空间小于阈值时，数据库会发出预警邮件，并在 online.log 中打印报警信息。

ONCONFIG 中 RESOURCE_CHECK 配置项和 DISK_THRESHOLD 配置项用于设定低限报警功能。

RESOURCE_CHECK 配置项负责启停会话限制模块。当 RESOURCE_CHECK 为 1 时，启动模块。当 RESOURCE_CHECK 为 0，关闭该模块。该参数改变需要重启数据库才可生效。

RESOURCE_CHECK 为 1 时，设置 DISK_THRESHOLD 参数，确定磁盘空间剩余量阈值（千分比），例如设置为 50 时，当根目录剩余空间低于千分之50时，数据库会发出预警邮件，并在 online.log 中打印报警信息。

【示例 11.1.29】

8.4 资源占用监控

GBase 8s 提供资源占用监控功能，无需专门配置。用于监控内存、磁盘等数据库资源使用情况。可通过 `onstat` 工具查看。

【示例 11.1.30】

9 安全审计

9.1 功能概述

GBase 8s 提供安全审计机制，对用户 in 数据库中的活动事件进行记录，该活动可能是修改或展示数据，或者是审计和安全策略的配置等。通过对这些活动的记录就可对可疑的活动进行分析。

为便于独立审计，设有专门的审计员进行审计管理，审计员可利用专门的审计操作界面对审计事件进行选择，查阅有关审计数据，处理报警信号。

9.2 审计事件

9.2.1 审计事件

GBase 8s 系统定义了28种审计事件来决定什么事件进行审计，如下表。通过审计工具对审计事件的设置，可以指定数据库服务或某个用户需要审计的事件组合。可以对成功或者失败的操作记录分别记录，或者两种同时记录。

序号	审计类型	审计事件	审计掩码
1	数据库管理类事件	创建数据库	CRDB
		删除数据库	DRDB
2	用户管理类事件	创建用户	CRUR
		删除用户	DRUR
		密码修改	PWUR
		信息修改	ALUR
		库特权授予	GRDB
		表特权授予	GRTB
		库特权回收	RVDB
		表特权回收	RVTB
3	用户登录类事件	用户登录	STSN
4	表管理类事件	创建表	CRTB
		删除表	DRTB
		修改表	ALTB
		创建索引	CRIX

序号	审计类型	审计事件	审计掩码
		删除索引	DRIX
		修改索引	ALIX
5	表操作类事件	插入数据	INRW
		删除数据	DLRW
		更新数据	UPRW
		读取数据	RDRW
		清空数据	TCTB
6	MAC 类事件	创建安全标签	CRLB
		删除安全标签	DRLB
		添加主体标记	GRLB
		回收主体标记	RVLB
7	审计分析类事件	异常检测	LCPM
		侵害分析	LKUR

9.2.2 配置说明

用户通过审计配置数据库审计事件，详细参见《GBase 8s 审计工具用户手册》中 3.2节“审计配置”。

9.3 审计日志

9.3.1 审计日志配置

ONCONFIG 文件 AUDIT_TYPE 配置项用来设定审计日志类型，值为 0 或 1。其中，设置为 1 时，数据库将审计信息存储系统表内，用户可以通过审计工具查看审计日志。设置为 0 时，日志信息将已数据文件形式存储到数据库所在服务器中，此时审计工具无法读取、分析审计日志。

9.3.2 审计日志一键迁出

GBase 8s 支持审计日志一键迁出功能，迁出审计日志后，用户可以根据需要选择是否清空审计数据库。

审计日志迁出工具为 sysaudit.sh，位于 /opt/gbase8s/etc/sysadmin 目录下。

该工具能够将审计数据库“sysadt”备份至用户输入目录，备份成功后，用户可

以使用 dbimport 工具，将所备份的审计数据库导入其他 GBase 8s 数据库中。

```
[root@build220 sysadmin]# sh sysaudit.sh
```

It will reset 'sysadt' database and reboot GBase8s Server, please back up the 'sysadt' database if necessary.

Finally, the server will switch to user mode.

Do you want to backup the 'sysadt' database(y/n) y

Input the backup dir for 'sysadt' :/opt/gbase8s

Back up 'sysadt' to /opt/gbase8s, please wait a moment.

.....

dbexport completed

Back up to /opt/gbase8s/sysadt.exp completely!

备注：此时 sysadt 数据库已备份至 /opt/gbase8s/sysadt.exp 目录

用户可使用 dbimport 工具，将 /opt/gbase8s/sysadt.exp 导入其他 GBase 8s 数据库中，查看审计信息。

Do you really want to reset 'sysadt' (y/n) y

备注：输入 y 表示清空审计数据库 sysadt，输入 n 表示不清空审计数据库 sysadt。

Starting reset 'sysadt' database , please wait a moment....

Space successfully dropped.

** WARNING ** A level 0 archive will need to be done before any chunks from DBspace sysauditdbs can be reused (see Database Server Administrator's manual).

Verifying physical disk space, please wait ...

Space successfully added.

** WARNING ** A level 0 archive of Root DBSpace will need to be done.

Reset 'sysadt' completely!

备注：“Reset 'sysadt' completely!”表示清空成功。

如果清空失败，请查看 \${GBASEBTDIR}/etc/sysadmin/sysaudit.log 日志，获取

相关信息。

9.4 审计关联分析

9.4.1 异常检测

当用户通过审计工具，开启“审计分析”中“异常检测”项，当用户访问其不具备CONNECT、RESOURCE、DBA 权限的数据库时，GBase 8s 将记录此事件至审计数据库中，可通过审计工具中“审计分析”-“异常检测”功能查看。

9.4.2 侵害分析

当用户通过审计工具，开启“审计分析”中“侵害分析”项，当用户连接数据库的连续失败次数大于阈值 CF_LIMIT_SWITCH 时，GBase 8s 将记录此事件至审计数据库中，可通过审计工具中“审计分析”-“潜在侵害”功能查看。

10 安全总开关及安全配置项汇总

10.1 功能概述

GBase 8s 提供安全模块总启动和关闭配置项。为 ONCONFIG 文件中 SEC_SWITCH 配置项，将 SEC_SWITCH 设置为1时，数据库启用安全模块，各子安全模块配置项生效（存储加密、通信加密除外）。将 SEC_SWITCH 设置为 0 时，数据库关闭安全模块（存储加密、通信加密除外）。

另外，GBase 8s 提供密码传输加密功能，无需对 Server 进行配置。

10.2 配置说明

配置文件	配置项	说明
ONCONFIG	SEC_SWITCH	1) 安全总开关，取值 0 或 1 2) 0: 关闭配置项 2-17 对应安全模块 3) 1: 启用配置项 2-17 对应安全模块，各子模块行为由各自参数控制
	CONF_SWITCH	1) 登录模式，取值 0、1、或 2 2) 0: 仅数据库内部用户可登录，如

配置文件	配置项	说明
		sysdba 3) 1: 仅操作系统用户可登录, 如root 4) 2: 支持操作系统用户和数据库内部用户 5) 详见 2.2 节
	PWDCOMPLEXCHK	1) 密码复杂度检查模块, 取值0或1 2) 0: 关闭模块 3) 1: 启用模块 4) 详见 3.2.2 节
	PWD_MIN_LENGTH	1) 最小密码长度, 取值8-32 2) 详见 3.2.2 节
	PWDDATECHK	1) 密码时效性检查模块, 取值0或1 2) 0: 关闭模块 3) 1: 启用模块 4) 详见 3.2.3 节
	CFPWDPERIOD	1) 密码更新最大周期, 取值 1-7 2) 详见 3.2.3 节
	CF_LIMIT_SWITCH	1) 尝试次数限制模块, 取值 0 或 1 2) 0: 关闭模块 3) 1: 启用模块 4) 详见 3.2.4 节
	CFLIMITNUM	1) 最大尝试失败次数, 取值0-5 2) 详见 3.2.4 节
	LOCKPERIOD	1) 用户被锁定后自动解锁时间(天), 取值1-1000 2) 详见 3.2.4 节
	SESTO_SWITCH	1) 超时认证模块, 取值 0 或 1 2) 0: 关闭模块 3) 1: 启用模块 4) 详见 3.2.6 节
	SES_TIMEOUT	1) 超时时间(秒), 取值1-2147483647

配置文件	配置项	说明
		2) 详见 3.2.6 节
	LBAC_SWITCH	1) 强制访问控制模块，取值0或1 2) 0：关闭模块 3) 1：启用模块 4) 详见 第 5 节
	SES_NUM_SWITCH	1) 会话限制模块，取值0或1 2) 0：关闭模块 3) 1：启用模块 4) 详见 8.2 节
	MAX_SESSION_NUM	1) 最大用户连接数，取值1-2147483647 2) 详见 8.2 节
	RESOURCE_CHECK	1) 资源预警模块，取值 0 或 1 2) 0：关闭模块 3) 1：启用模块 4) 详见 8.3 节和 8.4 节
	DISK_THRESHOLD	1) 根目录空间剩余量预警阈值，单位千分比 2) 详见 8.3 节和 8.4 节
	AUDIT_TYPE	3) 审计日志类型，取值 0 或 1 4) 0：日志文件模式，不支持审计工具查看审计日志、进行审计分析 5) 1：数据库文件模式，支持审计工具 6) 详见 9.3 节
	SERVER_ENCRYPTION_TYPE	1) 数据存储加密模块，取值 SOFT 或 CLEARTXT 2) SOFT：加密 3) CLEARTXT：不加密 4) 详见 6.1 节
	VPCLASS	1) 通信加密控制参数 2) 开启SSL需要增加 encrypt 类型，如 VPCLASS encrypt,num=1

配置文件	配置项	说明
	NETTYPE	1) 通信加密控制参数 2) 开启 SSL 需要增加 onsocssl 类型， 如 NETTYPE onsocssl, 1, 150, CPU
SQLHOSTS	onsocssl	1) 通信加密控制参数 2) 开启 SSL 需要增加 onsocssl 类型， 如 ol_gbase8s onsocssl 127.0.0.1 10603

11 使用示例

11.1 通用功能

11.1.1 数据库安装实例路径

```
/opt/gbase8s
```

11.1.2 获取环境变量

```
cd /opt/gbase8s  
source ol_gbase8s.ksh
```

11.1.3 启动实例

```
cd /opt/gbase8s  
source ol_gbase8s.ksh  
oninit -vy
```

11.1.4 停止实例

```
cd /opt/gbase8s  
source ol_gbase8s.ksh  
onmode -ky
```

11.1.5 企业管理器连接数据库实例（sysdba/GBase8szyj）

1. 以 root 用户进入路径 /opt/gbase8s_clienttool/ClientTools/Studio，双击 GBaseStudio
2. 在左侧“GBase 服务器”节点右键，选择“新建”--“服务器”，弹出新建服务器注册界面
3. 填写要访问实例的名称、数据库、实例名、用户名、密码等信息后，点击测试按钮
4. 如测试成功，则保存连接信息；如测试失败，检查填写信息是否有误
5. 双击已保存的连接，再次输入连接信息，即可连接到实例

11.1.6 安全管理器连接数据库实例（sysdsa/GBase8szyj）

1. 以 root 用户进入路径/opt/gbase8s_clienttool/ClientTools/MacTool，双击 MacTool
2. 在左侧“GBase 服务器”节点右键，选择“新建”--“服务器”，弹出新建服务器注册界面
3. 填写要访问实例的名称、数据库、实例名、用户名、密码等信息后，点击测试按钮
4. 如测试成功，则保存连接信息；如测试失败，检查填写信息是否有误
5. 双击已保存的连接，再次输入连接信息，即可连接到实例

11.1.7 安全管理器连接数据库实例（sysdaa/GBase8szyj）

1. 以 root 用户进入路径/opt/gbase8s_clienttool/ClientTools/AuditTool，双击 AuditTool
2. 在左侧“GBase 服务器”节点右键，选择“新建”--“服务器”，弹出新建服务器注册界面
3. 填写要访问实例的名称、实例名、用户名、密码等信息后，点击测试按钮
4. 如测试成功，则保存连接信息；如测试失败，检查填写信息是否有误
5. 双击已保存的连接，再次输入连接信息，即可连接到实例

11.1.8 创建数据库

1. sysdba 登录企业管理器，左树菜单右键单击“数据库”→“新建数据库”
2. 输入数据库名称 db1，其它默认
3. 点击“确定”，在左侧目录树，能看到新建的数据库db1节点

11.1.9 删除数据库

1. sysdba 登录企业管理器，左树菜单“数据库”→“db1”
2. 右键单击 db1，选择“删除”
3. 点击“确定”，在左侧目录树，新建的数据库 db1 节点已被删除

11.1.10 创建表

1. sysdba 连接企业管理器，连接数据库db1
2. 在表节点右键→新建表

3. 输入列名 c1, 选择 int 数据类型, 输入列名 c2, 选择 varchar 类型
4. 点击保存, 输入表名 t1
5. 在左树菜单中, 能够看到新建表 t1

11.1.11 修改表名

1. 左树菜单-数据库-dbl-t1, 右键单击 “t1”
2. 选择 “重命名” → 重命名为 t2
3. 点击 “确定”, 表名由 t1 变更为 t2

11.1.12 删除表

1. 左树菜单-数据库-dbl-t2
2. 右键单击 “t2” → “删除”
3. 点击 “确定”, t2 表删除不可见

11.1.13 创建视图

1. sysdba 登录企业管理器, 左树菜单选择 “数据库” - “dbl” - “视图”
2. 右键单击 “视图” 选择 “新建视图”
3. 输入视图名 “v_t1”, 结构体部分语句修改为 “AS SELECT * FROM t1; ”
4. 点击 “下一步”, 完成

11.1.14 删除视图

1. sysdba 登录企业管理器, 左树菜单选择数据库-dbl-视图
2. 右键单击 “视图” 选择 “删除”
3. 点击 “确定”, 视图 v_t1 被删除

11.1.15 查询表结构

1. sysdba 登录企业管理器, 左树菜单-数据库-dbl-t1
2. 右键单击 “t1” -> “设计表”
3. 能够显示正确的表结构

11.1.16 插入数据

插入合法数据时成功，插入不合法数据系统不添加数据且显示错误原因

1. sysdba 在企业管理器中选 db1，点击“新建查询”，在 SQL 编辑器中执行以下语句，授予 sysdsa 对表 t1 的 alter 权限：

```
grant alter on t1 to sysdsa;
```

2. sysdsa 登录安全管理器，连接数据库 db1，给用户 sysdba 和表 t1 打安全标签 lb11，使主体 sysdba 有权限访问客体 t1（详细步骤见附录）

3. 插入合法数据成功

sysdba 登录企业管理器，连接数据库 db1，新建查询，打开 SQL 编辑器执行以下语句，向表 t1 (c1 int, c2 varchar(50)) 插入数据成功

```
insert into t1 values(1,'a');  
Insert into t1 values(2,'b');
```

4. 向表中插入非法数据，操作失败且显示出错信息

如向表 t1 (c1 int, c2 varchar(50)) 插入以下数据失败，报错“字符型数据向数值型转换失败”

```
insert into t1 values('a','a');
```

11.1.17 修改数据

合法更新数据时，系统更新数据成功；不合法更新数据时不更新数据

1. sysdba 登录企业管理器，连接数据库 db1，新建查询，在 SQL 编辑器中执行以下语句，合法更新表中数据，操作成功

```
update t1 set c2='bb' where c1=2;
```

2. 不合法更新表中数据，操作失败

```
update t1 set c2='bb' where c1=4;
```

11.1.18 查询数据

查询符合条件的数据能检索成功；查询不符合条件数据时，系统返回空

1. sysdba 登录企业管理器，连接数据库 db1，新建查询，在 SQL 编辑器中执行以下语句，查询表中符合条件数据，操作成功

```
select * from t1 where c1=2;
```

2. 查询表中不符合条件数据，操作失败

```
select * from t1 where c1=3;
```

11.1.19 删除数据

合法删除数据时成功，非法删除数据系统不删除数据且报错显示原因

1. sysdba 登录企业管理器，连接数据库 db1，新建查询，在 SQL 编辑器中执行以下语句，合法删除表中数据，操作成功

```
delete from t1 where c1=1;
```

2. 非法删除表中数据，操作失败，报错提示未查询到 c1 为 3 的列

```
delete from t1 where c1=3;
```

11.1.20 修改表

增加合法字段时，字段增加成功；增加不合法字段时，系统不增加字段，显示出错原因

1. sysdba 登录企业管理器，左树菜单-数据库-db1-t1
2. 右键单击“t1”->“设计表”
3. 增加一列，列名输入 c3，数据类型选择 varchar(50)
4. 保存成功
5. 增加一列，列名输入 c1，数据类型选择 varchar(50)
6. 操作失败，报错列名已经存在

11.1.21 自主访问控制权限的赋予与回收

客体所有者规定并控制对客体的访问，并阻止非授权用户对客体的访问

1. sysdba 登录企业管理器，点击”新建查询“，打开 SQL 编辑器执行以下语句，创建用户 user1、user2，创建数据库 db1，创建表 t1，授予 sysdsa 对表 t1 的访问权限：

```
create user user1 with password 'GBase123' subject 'root' issuer
'GBASECA' ;

create user user2 with password 'GBase123' subject 'root' issuer
'GBASECA' ;

drop database if exists db1;
create database db1 with log;
create table t1(c1 int,c2 varchar(50));

grant alter on t1 to sysdsa;
```

2. sysdsa 连接库 db1，登录安全管理器，给用户 sysdba、user1、user2、user3

和表 t1 均打相同标签 lb11;

3. sysdba 登录企业管理器，连接库 db1，新建查询，向表中插入数据 (1, 'a')，授予用户 user1、user2 对库 db1 的 connect 权限，收回 public 用户对表 t1 的所有权限，授予用户 user1 对表 t1 的 select 权限：

```
insert into t1 values(1,' a' );  
grant connect to user1;  
grant connect to user2;  
revoke all on t1 from public;  
grant select on t1 to user1;
```

4. user1 登录企业管理器，新建查询，user1 能正常查询表 t1

```
select * from t1;
```

5. user2 登录企业管理器，并尝试查询表t1，报错没有 select 权限：

```
select * from t1;
```

11.1.22 权限传播机制

数据库管理系统应具有授权传播机制

1. user1 登录企业管理器，尝试将对表 t1 的 select 权限授予 user2, 授权失败，报错没有 GRANT 权限：

```
grant select on t1 to user2;
```

2. sysdba 登录企业管理器，授予 user1 对表 t1 的 select 权限及向其他用户授权的权限：

```
grant select on t1 to user1 with grant option;
```

3. user1 登录企业管理器，再次尝试授予 user2 对表 t1 的 select 权限，授权成功

```
grant select on t1 to user2;
```

4. user2 登录企业管理器，user2 能正常查询表 t1 中的数据：

```
select * from t1;
```

11.1.23 标记功能

1. sysdba 登录企业管理器，新建查询，创建数据库 db1，创建表 t1，并授予 sysdsa 用户对表 t1 的 alter 权限：

```
drop database if exists db1;  
create database db1 with log;  
create table t1(c1 int,c2 varchar(50));
```

```
grant alter on t1 to sysdsa;
```

2. sysdsa 登录安全管理器，连接数据库 db1
3. 双击左树的“范畴管理”标签页，点击左下角“标识范畴”，输入范畴名 fc1，选择范畴1（下拉菜单）
4. 双击左树的“标签管理”标签页，点击左下角“创建安全标签”，输入安全标签名 lb11，选择密级1，范畴 fc1，点击确定
5. 双击左树的“主体管理”标签页，点击左下角“标识主体”，输入主体名 sysdba，选择安全标签 lb11，点击确定
6. 双击左树的“客体管理”标签页，点击左下角“标识客体”，输入客体名 t1，给每列打安全标签 lb11，点击确定
7. sysdba 登录企业管理器，连接 db1，新建查询，向表 t1 中插入数据：

```
insert into t1 values(1,'a');  
insert into t1 values(2,'b');
```

11.1.24 标记的粒度

强制访问控制范围限定在所定义的主体和客体中，粒度是表级、记录或字段级；主体可以读/写其具有相关权限的客体，不可以读/写其不具有相关权限的客体

1. sysdba 登录企业管理器，新建查询，创建数据库 db1，创建表 t1，并授予 sysdsa 对表 t1 的 alter 权限

```
create database db1 with log;  
create table t1(c1 int,c2 int,c3 int,c4 int,c5 int,c6 int,c7 int,c8 int,c9 int);  
grant alter on t1 to sysdsa;
```

2. sysdsa 登录安全管理器，给 sysdba 用户打安全标签 lb212（密级2，范畴1,2），给表 t1 的每个列打安全标签，c1 列标签为 lb31（密级3，范畴1），c2 列标签为 lb3123（密级3，范畴1,2,3），c3 列标签为 lb312（密级3，范畴1,2），c4 列标签为 lb11（密级1，范畴1），c5 列标签为 lb1123（密级1，范畴1,2,3），c6 列标签为 lb112（密级1，范畴1,2），c7 列标签为 lb21（密级2，范畴1），c8 列标签为 lb2123（密级2，范畴1,2,3），c9 列标签为 lb212（密级2，范畴1,2）
3. sysdba 登录企业管理器，连接数据库 db1，新建查询，对表 t1 的每列进行增删改查（读写）操作
 - 1) 主体密级大于客体密级，且主体范畴包含客体范畴时，主体（sysdba）对客体（表的 c1 列）可读不可写
 - 2) 主体密级大于客体密级，且主体范畴不包含客体范畴时，主体（sysdba）对客体（表的 c2 列）无读写权限

- 3) 主体密级大于客体密级时，且主体范畴等于客体范畴时，主体（sysdba）对客体（表的 c3 列）可读不可写
- 4) 主体密级小于客体密级，且主体范畴包含客体范畴时，主体（sysdba）对客体（表的 c4 列）无读写权限；
- 5) 主体密级小于标签密级，且主体范畴不包含客体范畴时，主体（sysdba）对客体（表的 c5 列）无读写权限
- 6) 主体密级小于客体密级，且主体范畴等于客体范畴时，主体（sysdba）对客体（表的 c6 列）无读写权限
- 7) 主体密级等于客体密级，且主体范畴包含客体范畴时，主体（sysdba）对客体（表的 c7 列）有读写权限
- 8) 主体密级等于客体密级，且主体范畴不包含客体范畴时，主体（sysdba）对客体（表的 c8 列）无读写权限
- 9) 主体密级等于客体密级，且主体范畴等于客体范畴时，主体（sysdba）对客体（表的 c9 列）有读写权限

11.1.25 标记的输入与输出

对于从数据库管理系统安全子系统控制范围外输入的未标记客体，应进行默认标记或由安全保密管理员进行标记

前提：t1 表已带有标签

1. server 端切换到 root 用户模式下，将数据库 db1 导出到 dbexport1 文件夹下：

```
su gbasedbt
cd /opt/gbase8s
source ol_gbase8s.ksh
onmode -wf CONF_SWITCH=1
mkdir dbexport1
chown gbasedbt:gbasedbt dbexport1
chmod 777 dbexport1
dbexport db1 -o dbexport1
```

2. 打开 dbexport.out，查看表 t1 信息是否带有安全标签 lb11
3. server 端执行 onmode -wf COND_SWITCH=0 切出 root 用户，sysdba 登录到企业管理器，删除数据库 db1
4. server 端切换到 root 用户模式下，并进入 dbexport1 目录中，导入 db1 数据库数据

```
onmode -wf COND_SWITCH=1
```

```
cd /opt/gbase8s/dbexport1
dbimport db1 -lbac
```

5. server 端执行 `onmode -wf COND_SWITCH=0` 切出 root 用户, sysdsa 连接 db1 数据库, 登录安全管理器, 查看客体管理, t1 的标签信息恢复成功
6. sysdba 登录企业管理器, 连接库 db1, 新建查询, 查看表 t1 已恢复成功
`select * from t1;` (返回2行)

11.1.26 备份与恢复

1. 数据库中应具有完全（增量）备份恢复功能

【准备】

```
cd /opt/gbase8s
mkdir backups
chmod 777 backups
chown gbasedbt:gbasedbt backups
onmode -wf TAPEDEV=/opt/gbase8s/backups
onmode -wf LTAPEDEV=/opt/gbase8s/backups
onmode -wf BAR_BSALIB_PATH=/opt/gbase8s/lib/libbsapsm.so
export BAR_BSALIB_PATH=/opt/gbase8s/lib/libbsapsm.so
onmode -wf PSM_CATALOG_PATH=/opt/gbase8s/etc/psm
```

- 1) 准备数据 (sysdba 与表 db3:t1 符合强访策略)

```
drop database if exists db3;
create database db3 with log;
create table t1(id int);
insert into t1 values(1);
```

- 2) 进行 0 级备份

```
onbar -b -w -L 0
```

输入用户名 sysdba

- 3) 备份完成后, 删除数据库 db3
- 4) 关闭数据库实例, 进行0级恢复

```
onbar -r
```

- 5) 验证数据恢复成功:

```
connect to 'db3@ol_gbase8s' user 'sysdba';
select * from t1; --返回 1 行(1)
```

- 6) 以 sysdba 身份再次插入数据


```
insert into t1 values(2);
```

7) 进行 1 级备份

```
onbar -b -L 1
```

输入用户名 sysdba

8) 关闭数据库实例, 进行1级恢复

```
onmode -ky
```

```
onbar -r
```

查询表 t1 中的数据:

```
select * from t1; --返回2行(1) (2)
```

2. 数据库管理系统应对备份与恢复操作设置权限

检查各用户权限, 查看备份权限是否被严格限制

1) 具备备份恢复权限的授权用户执行备份还原操作, 查看是否成功, 如root 用户;

2) 以不具备备份权限的用户登录, 执行备份还原操作, 查看是否失败, 如新建用户 user1。

3. 备份数据加密

以授权用户登录, 对数据进行备份操作, vi备份数据文件为密文存储

4. 数据库管理系统应具有备份数据完整性保护机制

以授权用户身份分别执行备份操作, 使用 vi 打开并篡改备份文件, 使用修改后的备份文件进行还原操作, 操作失败

11.1.27 客体重用

产品禁止客体重用

1. sysdba 登录企业管理器, 点击“新建查询”, 在 SQL 编辑器中执行以下语句, 建库 db4, 建表 t1, 创建用户 user1;

```
drop database if exists db4;
create database db4 with log;
create table t1(id int);
create user user1 with password 'GBase123' subject 'root' issuer
'GBASECA';
```

2. sysdba 登录企业管理器, 选中 db4, SQL 编辑器中执行以下语句, 赋予 user1 访问 db4 中 t1 的权限, 并查看权限表, user1 对表 t1 有访问权限

```
grant connect to user1;
```

```
revoke all on t1 from public;  
grant select on t1 to user1;  
select tabname, grantee from systabauth a, systables b where a.tabid>99  
and a.tabid=b.tabid;
```

3. 企业管理器中，选中db4，新建查询，SQL编辑器中执行以下语句，删除表t1，再次查看 user1 权限表，关于db4 中t1 的权限已被收回；

```
drop table t1;  
select tabname, grantee from systabauth a, systables b where a.tabid>99  
and a.tabid=b.tabid;
```

4. 企业管理器中，选中 db4，新建查询，SQL 编辑器中执行以下语句，再次创建 t1 ，此时查看 user1 权限表，user1对t1 无访问权限

```
create table t1(id int);  
select tabname, grantee from systabauth a, systables b where a.tabid>99  
and a.tabid=b.tabid;
```

5. 以 user1 身份登录企业管理器，选中 db4，点击“新建查询”，在 SQL 编辑器中执行以下语句，user1 对 t1 无访问权限：

```
select * from t1;
```

11.1.28 会话机制（最大连接数）

1. 设置会话最大连接数为 2（onconfig 文件中 MAX_SESSION_NUM 设为 2），重启数据库实例
2. 开启 3 个 session，执行“dbaccess - -”，通过执行 connect 语句连接实例：

```
connect to '@ol_gbase8s' user 'sysdba';
```

3. 前两个 session 中 sysdba 输入正确密码可以成功登录，第三个 session 输入正确密码，连接失败，提示已经超出连接限制。
4. 设置会话最大连接数为 3（onconfig 文件中 MAX_SESSION_NUM 设为 3），重启数据库实例

3个 session 可同时连接实例

11.1.29 低限报警

产品应具有服务器存储空间低限报警功能

1. 开启邮件通知。配置 etc/alarmprogram.sh 中邮件相关参数：

```
ALARMADMIN=1
```

```
ADMINEMAIL=root@localhost.localdomain
```

```
MAILUTILITY=mail
```

2. 在数据库实例配置文件 `onconfig` 中，修改参数 `DISK_THRESHOLD` 为预警值（以990为例），即磁盘剩余空间如果小于预警值%，数据库将会发出预警邮件
3. 重启 `server`
4. 磁盘剩余空间小于 990% 时，GBase 8s 将会发出预警邮件，并且 `online.log` 中会打印预警信息。

11.1.30 资源占用监控

1. 在 `/opt/gbase8s` 路径下，执行 `source ol_gbase8s.ksh`
2. 执行 `onstat -d`，查看数据库 DBSpace 占用情况
3. 执行 `onstat -g mem`查看数据库内存占用情况

11.1.31 导入文件类型筛选

1. `sysdba` 登录企业管理器，点击“新建查询”，打开 SQL 编辑器执行以下语句，创建数据库 `db5`，创建表 `t1`：

```
drop database if exists db5;
create database db5 with log;
create table t1(id int,name varchar(50));
```

2. 双击左树菜单数据库库-db5-表-t1，右键单击“t1”，选择“导入表数据”，在弹出的导入文件窗口右下角，有多种导入文件类型可进行筛选：`*.csv`、`*.txt`、`*.xml`、`*.xls`、`*.xlsx`

11.2 数据库安全功能

11.2.1 管理员身份鉴别

1. 支持多种身份鉴别方式

`sysdba`、`sysdsa`、`sysdaa` 登录企业管理器、安全管理器、审计工具时均采用“用户名+口令+证书”相结合的方式进行验证

2. 只有输入正确的管理员身份鉴别信息，方可通过身份鉴别

1) `sysdba`、`sysdsa`、`sysdaa` 分别登录企业管理器、安全管理器、审计工具，输入正确的口令和正确的证书，登录成功；

- 2) 输入错误的口令和正确的证书，登录失败；
- 3) 输入正确的口令和错误的证书，登录失败；
3. 确保身份标识符在此系统生命周期内用户唯一性

- 1) 创建用户 user1：以 sysdba 登录，执行如下语句：

```
create user user1 with password 'GBase123' subject 'root' issuer
'GBASECA' ;
```

- 2) 删除用户 user1：

```
drop user user1;
```

- 3) 再次创建用户 user1 失败, user1 用户在此系统生命周期内用户唯一：

以 sysdba 登录，执行如下语句失败

```
create user user1 with password 'GBase123' subject 'root' issuer
'GBASECA' ;
```

4. 数据库管理系统应显示管理员显示日期、时间、来源、上次登录系统情况、上次成功访问系统以来身份鉴别失败的情况和口令到期天数

11.2.2 口令要求

管理员口令长度可设置，口令至少为 8 位，复杂度至少为字母数字组合

1. server 端分别修改 sysdba、sysdsa、sysdaa 管理员口令，修改为不符合长度、复杂度要求的口令，修改失败；

```
onmode -wf CONF_SWITCH=1;
dbaccess sysuser -
alter user sysdba modify passowrd 'gbase123' ;
alter user sysdsa modify passowrd 'gbaseabc' ;
alter user sysdaa modify passowrd 'GBase12' ;
```

2. 修改为符合长度、复杂度要求的口令，修改成功；

```
alter user sysdba modify passowrd 'GBase123' ;
alter user sysdsa modify passowrd 'GBase123' ;
alter user sysdaa modify passowrd 'GBase123' ;
```

11.2.3 用户尝试次数限制功能

具有用户身份鉴别尝试次数限制功能，尝试次数应可设定，且至多为5次

1. sysdba、sysdsa、sysdaa 分别登录企业管理器、安全管理器、审计工具，均连续 5 次输入错误的口令，均登录失败，查看 server 端 online.log 提

- 示账号被锁定；
2. 超过设定的用户锁定时间后(锁定时间为配置文件中 LOCKPERIOD 参数值)，数据库自动解锁用户
 3. sysdba、sysdsa、sysdaa 重新登录企业管理器、安全管理器、审计工具，输入正确的口令和证书，登录成功

11.2.4 口令更换周期

具有用户定期更换口令功能，更换周期不得长于一周

1. 超过 7 天使用同一口令，管理员（sysdba、sysdsa、sysdaa）登录管理界面登录失败，查看 server 端 online.log 提示更改口令；
2. Server 端切换至 root 用户，分别修改 sysdba、sysdsa、sysdaa 管理员口令：

```
onmode -wf CONF_SWITCH=1;
dbaccess sysuser -
alter user sysdba modify passowrd 'Gbase8szyj' ;
alter user sysdsa modify passowrd 'Gbase8szyj' ;
alter user sysdaa modify passowrd 'Gbase8szyj' ;
onmode -wf CONF_SWITCH=0;
```

3. 管理员（sysdba、sysdsa、sysdaa）重新登录，登录成功

11.2.5 超时认证

用户登录管理界面，长时间（SESTO_SWITCH 设定值）空闲不进行操作；再次进行操作时，需要输入管理员信息重新登录

11.2.6 管理员权限划分

系统具有相互独立、相互制约的系统管理员、安全保密管理员和安全审计员三个管理员角色。管理员权限实施了分立，不应存在超级管理员或权限过高的角色和用户。

- 1) 数据管理员主要负责生成用户身份标识符和数据维护；
- 2) 安全保密管理员主要负责标签设定、以及安全审计员日志的审查分析；
- 3) 安全审计员主要负责对数据管理员和安全保密管理员的日志进行审查分析

11.2.7 审计日志一键迁出

能够对管理员和用户的操作行为和登录行为进行日志记录

1. sysdaa 登录审计工具，菜单栏-系统-审计规则配置，勾选 DB 管理类事件（创建数据库），勾选用户管理类事件（创建用户和库特权授予），勾选用户登录类事件（用户登录），勾选表管理事件（创建表），勾选表操作事件（插入数据、读取数据），勾选 MAC 管理类事件（创建安全标签，添加主体标记），点击确定
2. sysdba 登录企业管理器，新建查询，创建用户 user1、user2，建库 db1，建表 t1，授予 user1 对库 db1 的 connect 权限，并授予 sysdsa 对表 t1 的 alter 权限

```
create user user1 with password 'GBase123' subject 'root' issuer
'GBASECA' ;

create user user2 with password 'GBase123' subject 'root' issuer
'GBASECA' ;

create database db1 with log;
create table t1(c1 int);
grant connect to user1;
grant connect to user2;
revoke all on t1 from public;
grant select on t1 to user1 with grant option;
grant alter on t1 to sysdsa;
```

3. sysdsa 登录安全管理器，创建安全标签 lb11，并给用户 sysdba、user1、user2 和表 t1 打标签 lb11
4. sysdba 登录企业管理器，新建查询，向表中插入数据

```
insert into t1 values(11);
```

5. user1 登录企业管理器，选中库 db1，新建查询，查询表 t1，并将对表 t1 的 select 权限授予 user2:

```
select * from t1;
grant select on t1 to user2;
```

6. user2 登录企业管理器，选中库 db1，新建查询，可查询表 t2

```
select * from t1;
```

7. sysdaa 登录审计工具，双击左树菜单-审计查看，查看以上操作的审计记录：
 - 1) 在用户登录审计下，分别双击 user1、sysdba、sysdsa，可看到管理员和用户登录的审计记录；
 - 2) 在用户管理审计下，双击创建用户，可看到 sysdba 创建用户 user1、user2 的审计记录；双击授予权限，可看到 sysdba 对 user1、user2 访问 db1 的库特权授予的审计记录
 - 3) 在 MAC 管理审计下，双击安全标签维护，可看到 sysdsa 创建安全标签记录；

双击主体标记维护，可看到 sysdsa 对主体和客体的打标签记录

- 4) 在 DB 管理审计下，双击创建数据库，可看到创建库 dbdb1 的审计记录
- 5) 在表管理审计下，双击穿件表，可看到创建表的审计记录
- 6) 在表操作审计下，双击插入数据和查询数据，可看到插入和查询的审计记录

11.2.8 审计策略设置

1. sysdaa 用户登录审计工具，菜单栏-系统-审计规则配置，在弹出的审计配置窗口中，勾选想审计的事件，如选择 DB 管理类事件“CRDB(创建数据库)”和“DRDB(删除数据库)”-确定

2. sysdba 登录企业管理器，新建查询，创建数据库 db1，随后删除：

```
create database db1 with log;  
drop database db1;
```

3. sysdaa 用户登录审计工具，双击左树菜单-审计查看-DB管理审计，双击创建数据库和删除数据库，可看到创建和删除库 db1 的记录

11.2.9 关联分析

数据库具备基于审计的潜在侵害分析能力

1. sysdaa 登录审计工具，菜单-系统-审计规则配置，弹出审计配置对话框，勾选选择审计分析类事件（侵害分析），确定
2. sysdba 尝试登录企业管理器，连续 5 次输入错误的口令，登录失败
3. sysdaa 登录审计工具，双击左树菜单-审计查看-侵害分析-潜在侵害，审计记录中可见锁定用户的记录
4. 超过设定的用户锁定时间后（锁定时间为配置文件中 LOCKPERIOD 参数值），数据库自动解锁用户

11.2.10 日志数据安全

日志数据存储空间将满时有告警提示，为防止审计数据丢失，server 自动忽略可审计事件，存储空间恢复后可正常记录审计事件。

1. sysdaa 登录审计工具，进入“审计查看-用户登录审计-sysdaa”，查看 sysdaa 刚刚的审计记录
2. Server 端 onconfig 文件中修改 DISK_THRESHOLD 参数值为 999，重启 server。当磁盘剩余空间小于 999% 时，数据库 online.log 中会打印预警信息，登录客户端管理器，会弹出告警界面。

3. sysdaa 重新登录审计工具，进入“审计查看-用户登录审计-sysdaa”，不记录本次 sysdaa 登录信息
4. Server 端 onconfig 文件中修改 DISK_THRESHOLD 参数值为 50，重启 server
5. sysdaa 重新登录审计工具，进入“审计查看-用户登录审计-sysdaa”，记录本次 sysdaa 登录信息

12 附录

12.1 附录1-GBase 8s 安全通讯SSL/TLS配置指南

12.1.1 证书准备

如果GBase 8s产品需要使用的证书为客户提供，可直接调过本章，从第二章的如何设置服务端及客户端开始您的SSL/TLS之旅。

如果希望尝试从零开始从生成证书开始GBase 8s的SSL/TLS之旅，那么首先需要安装第三方SSL库并以此来创建对应的证书，本章将讲述如何使用第三方（开源）SSL库并生成证书。

12.1.2 开源SSL工具的使用

本章讲述如何使用第三方SSL工具来构建您的SSL/TLS通讯所使用的证书。

12.1.2.1. 使用OpenSSL

OpenSSL的下载地址如下：

<https://www.openssl.org/source/>

在本文后续示例中，使用的是OpenSSL 3.2.2，为了支持更加安全的TLS 1.3协议，建议您使用不低于OpenSSL 3.0的版本或更新的版本。

下载OpenSSL源码后，您需要对源码进行编译及安装，方法如下：

进入到openssl源码根目录，以将openssl安装到“/opt/openssl_3.2.2”为例，执行如下操作：


```
./config --prefix=/opt/openssl_3.2.2 -shared  
  
make  
  
make install
```

拷贝这两个文件到系统路径，以供其他程序或应用使用（例如生成证书）：

```
cp /opt/openssl_3.2.2/lib64/libcrypto.* /lib64/  
  
cp /opt/openssl_3.2.2/lib64/libssl.* /lib64/
```

注：libssl.so.* 和libcrypto.so.* 在不同版本的openssl的后缀有可能不同，编译完成后，这两个库的位置在上一步编译输出prefix指定的目录的lib64下，请根据实际情况进行拷贝。

12.1.2.2. 使用铜锁ssl

铜锁SSL为蚂蚁集团开源的项目。基于 OpenSSL 3.0 修改而来，项目前身是 BabaSSL，现已改名为铜锁/Tongsuo。

本章示例中使用的是铜锁8.5.0(该版本基于OpenSSL 3.0.3)，为了保证安全性，建议选择的版本不要低于该版本。

下载地址：

```
https://gitee.com/babassl/Tongsuo/tree/master
```

帮助文档地址：

```
https://www.yuque.com/tsdoc/ts
```

使用ntls（0024标准或TLCP）

```
https://babassl.readthedocs.io/zh/latest/Tutorial/SM/ntls/
```

编译及安装：

进入到tongsuo源码根目录，以输出到/opt/tongsuo_8.5.0 为例，执行如下操作：

```
./config --prefix=/opt/tongsuo_8.5.0 -Wl,-rpath,/opt/tongsuo_8.5.0  
  
enable-ec_elgamal enable-paillier enable-ntls  
  
perl configdata.pm --dump  
  
make -j
```

```
make install
```

说明： 如果不使用tongsuo支持的ntls或paillier算法等特性，铜锁的编译和安装方式可以同openssl完全一致。以上示例中的“enable-ntls”、“enable-ec_elgamal”等配置参数目的即为打开这些特性。

拷贝这两个文件到系统路径，以供其他项目使用（例如生成证书）：

```
libssl.so.3 => /lib64/libssl.so.3 (0x00007eff41854000)
```

```
libcrypto.so.3 => /lib64/libcrypto.so.3 (0x00007eff41214000)
```

使用tongsuo开源库的优点为支持ntls，ntls为GM/T 0024 SSL VPN和TLCP协议的安全通信协议，其特点是采用加密证书/私钥和签名证书/私钥相分离的方式。详细使用ntls协议的方法请参阅如下地址：

```
https://babassl.readthedocs.io/zh/latest/Tutorial/SM/ntls/
```

12.1.2.3. 其他开源ssl工具库

如下两个工具也可以完成证书的生成操作，由于同现有GBase 8s 使用的openssl差异较大，请自行查阅对应产品文档。

- **tongsuo-mini**

铜锁迷你版（tongsuo-mini），简称“迷你锁”，是一个轻量级的密码学算法库，专为嵌入式系统和物联网设备等资源受限场景提供常用的密码学算法和安全传输协议，并可以适应超低内存和存储的极限要求。“迷你锁”通过高度模块化，允许用户在编译时只开启需要的功能，不浪费存储空间。同时，通过紧凑的内存对齐等方式，压缩运行时内存。

下载地址：

```
https://github.com/Tongsuo-Project/tongsuo-mini
```

- **GmSSL-3.1.1**

GMSSL：北京大学开源的项目。原版基于 OpenSSL 1.0.2 修改而来。新的 GMSSL 3.0 基本上是重新开发，跟 OpenSSL 的目录结构差别很大。

下载地址：

```
https://github.com/guanzhi/GmSSL/releases
```

12.1.3 证书生成

完成工具链（openssl或铜锁ssl）的编译和安装后，即可使用该工具完成GBase 8s通讯ssl所需要的证书了。为了方便使用，本文档将对应的生成证书脚本放置在文档附件内，在本文档内仅讲述如何调用该脚本。生成证书的原理及详细参数，请自行向所使用的开源ssl工具网站或论坛内学习。

12.1.3.1. 生成标准证书

如果您已经拥有了数据库服务所需的证书，则请跳过本章。

如果您打算直接使用已构建好的证书，请直接使用“附件A：示例证书”。

本节将讲述如何使用开源SSL工具构建证书的方法。由于openssl 3.2和铜锁8.4生成证书的方式完全相同。本节中的示例将通过OpenSSL 3.2.0版本工具来展示如何生成证书。

注：本示例生成的证书包括CA、8s服务端、8s客户端用户证书。其中8s客户端用户证书包括gbasedbt、sysdba、sysdaa、ysdsa、user1、user2、user3这些用户所对应使用的证书。

具体步骤如下：

1、 设定 ssl工具库所在路径（上一章节的prefix路径）

注： 使用不同ssl工具库都可以生成sm2证书，但需要确保“/lib64”下的libssl和libcrypto这两个库和使用的ssl工具库相匹配。

openssl 3.2库路径：

```
export PATH=/opt/openssl_3.2.2/bin:$PATH
```

或者 铜锁 8.4库路径：

```
export PATH=/opt/tongsuo_8.5.0:$PATH
```

2、 生成CA证书

创建CA证书私钥

```
openssl genpkey -algorithm RSA \  
    -pass pass:111111 \  
    -out "./certs/CA.key"
```

请求证书

```
openssl req \  
  
    -new -sha256 \  
  
    -key "./certs/CA.key" \  
  
    -passin pass:111111 \  
  
    -out "./certs/CA.csr" \  
  
    -subj  
  
"/C=CN/ST=TIANJIN/O=GBASE/OU=GBASE8S/CN=GBASECA"  
  
# 自签署证书  
  
openssl x509 \  
  
    -req \  
  
    -days 36500 \  
  
    -sha256 \  
  
    -extfile /etc/pki/tls/openssl.cnf \  
  
    -extensions v3_ca \  
  
    -passin pass:111111 \  
  
    -signkey "./certs/CA.key" \  
  
    -in "./certs/CA.csr" \  
  
    -out "./certs/CA.crt"
```

3、生成8s Server端证书

创建服务器私钥

```
openssl genpkey -algorithm RSA \  
  
    -pass pass:111111 \  
  
    -out "./certs/server.key"
```

请求证书

```
openssl req \  

```

```
-new \  
  
-sha256 \  
  
-passin pass:111111 \  
  
-key "./certs/server.key" \  
  
-out "./certs/server.csr" \  
  
-subj "/C=CN/ST=TIANJIN/O=GBASE/OU=GBASE8S/CN=GBase8s"
```

使用CA证书签署服务器证书

```
openssl x509 \  
  
-req \  
  
-days 36500 \  
  
-sha256 \  
  
-extfile "/etc/pki/tls/openssl.cnf" \  
  
-extensions v3_req \  
  
-passin pass:111111 \  
  
-CA "./certs/CA.crt" \  
  
-CAkey "./certs/CA.key" \  
  
-CAserial "./certs/CA.srl" \  
  
-CAcreateserial \  
  
-in "./certs/server.csr" \  
  
-out "./certs/server.crt"
```

4、生成8s客户端对应证书

```
user_list=(gbasedbt sysdba sysdaa sysdsa user1 user2 user3)  
  
for user_item in ${user_list[@]}; do  
  
    # 生成客户端私钥  
  
    openssl genpkey -algorithm RSA \  

```

```
-pass pass:111111 \
-out "./certs/${user_item}.key"

# 申请证书

openssl req \
    -new \
    -sha256 \
    -passin pass:111111 \
    -key "./certs/${user_item}.key" \
    -out "./certs/${user_item}.csr" \
    -subj
"/C=CN/ST=TIANJIN/O=GBASE/OU=GBASE8S/CN=${user_item}"

# 使用CA证书签署客户端证书

openssl x509 \
    -req \
    -days 36500 \
    -sha256 \
    -extfile "/etc/pki/tls/openssl.cnf" \
    -extensions v3_req \
    -passin pass:111111 \
    -CA "./certs/CA.crt" \
    -CAkey "./certs/CA.key" \
    -CAserial "./certs/CA.srl" \
    -CAcreateserial \
    -in "./certs/${user_item}.csr" \
```

```
-out "/certs/${user_item}.cert"

done
```

详细内容请参阅“附件B：生成RSA证书”。

12.1.3.2. 生成国密算法证书

SM2算法证书同RSA证书生成流程基本一致，区别在于生成私钥时算法指定为SM2，同时，生成证书请求和证书文件时指定使用的签名算法为SM3（RSA证书的杂凑算法使用的是SHA256）。

在下表中，以生成server证书为例展示在创建证书时使用两种算法的差异，表左侧为生成RSA算法证书，右侧为SM2算法证书，红色部分则为这两种证书生成方式的差异。

# 创建服务器私钥		# 创建服务器私钥
openssl genpkey	<	openssl genpkey -algorithm ec \
-algorithm RSA \	>	
		-pkeyopt ec_paramg
		en_curve:sm2 \
-pass pass:111111 \	=	-pass pass:111111 \
-out		-out
"/certs/server.key"		"/certs/server.key"
# 请求证书		# 请求证书
openssl req \		openssl req \
-new \		-new \
-sha256 \	<	-sm3 \
	>	
-passin pass:111111	=	-passin pass:111111
\		\
-key		-key

"/certs/server.key" \	"/certs/server.key" \
-out	-out
"/certs/server.csr" \	"/certs/server.csr" \
-subj	-subj
"/C=CN/ST=TIANJIN/O=GBASE/OU=G	"/C=CN/ST=TIANJIN/O=GBASE/OU=G
BASE8S/CN=GBase8s"	BASE8S/CN=GBase8s"
# 使用CA证书签署服务器证	# 使用CA证书签署服务器证
书	书
openssl x509 \	openssl x509 \
-req \	-req \
-days 36500 \	-days 36500 \
-sha256 \	-sm3 \
-extfile	-extfile
"/etc/pki/tls/openssl.cnf" \	"/etc/pki/tls/openssl.cnf" \
-extensions	-extensions
v3_req \	v3_req \
-passin pass:111111	-passin pass:111111
\	\
-CA "/certs/CA.crt"	-CA "/certs/CA.crt"
\	\
-CAkey	-CAkey
"/certs/CA.key" \	"/certs/CA.key" \
-CAserial	-CAserial
"/certs/CA.srl" \	"/certs/CA.srl" \

-CAcreateserial \	-CAcreateserial \
-in	-in
"/certs/server.csr" \	"/certs/server.csr" \
-out	-out
"/certs/server.crt"	"/certs/server.crt"

详细内容请参阅“附件C：生成SM2证书”

12.1.3.3. 查看证书信息

可直接使用openssl来查看证书的issuer和subject信息

命令如下：

```
openssl x509 -issuer -subject -in ./certs/gbasedbt.crt
```

例如以此命令查看生成的gbasedbt的证书，展示该证书的分发者和拥有者的信息如下：

```
[root@localhost gen_cert]# openssl x509 -issuer -subject -in ./certs/user1.crt
issuer= /C=CN/ST=TIANJIN/O=GBASE/OU=GBASE8S/CN=GBASECA
subject= /C=CN/ST=TIANJIN/O=GBASE/OU=GBASE8S/CN=user1
-----BEGIN CERTIFICATE-----
MIIDjjCCAnagAwIBAgIUUugtqYoEtyuFoS5cZz82bywAqhswDQYJKoZIhvcNAQ
EL
BQAwUzELMAkGA1UEBhMCQ04xEDAOBgNVBAgMB1RJQU5KSU4xDjAMBgN
VBAoMBUdC
QVNFMRAdDgYDVQQLEdHQkFTRThTMRAwDgYDVQQDDAdHQkFTRUNB
MCAXDTI0MDcy
MjA1MjcxOFoYDzIxMjQwNjI4MDUyNzE4WjBRMQswCQYDVQQGEwJDTjEQM
A4GA1UE
CAwHVEIBTkpJTjEOMAwGA1UECgwFR0JBUEUxEDAOBgNVBAsMB0dCQVNF
FMxDjAM
```

BgNVBAMMBXVzZXIxMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQE
Au6ht
BPAqCs0XiZG3JtTcAV2Bch/V9w4d5qar58gEfkhdxKUTJ3hhbiBGQatbli541N44
ZOOFX2kXjBWXdnyY8xwm0hRK4MHNhAAzF67y2tEaxXRkAgSkSIhFKpuRiaglf
c43
+xgb1b7MG2TqaHh7RfQJhYoZ4Jz+TSO6zE8yJgn2dyoh9w0WJ5ShyI5Rdeuu
WHZu
YV2wZekTgDYdw7qTRb7nunX+YUBu9+XXmHS8Mn8V7fN3oKY4VR1BatFv+tE
K2Vd3
QKIqUXzGPNSknOdniWJYw4Sgbmkwfo8ts66WmdBOP4yAWDsMoTWW7fzp
3nTNyny
bDSPrNIXnCdAs8xqyQIDAQABo1owWDAJBgNVHRMEAjAAMAsGA1UdDwQE
AwIF4DAd
BgNVHQ4EFgQUgM3FW2fLh+jIIG/vwIwMc3D/4OwwHwYDVR0jBBgwFoAUYv
ZgPdWC
QdFQMKd9WBIdALZMxdkwDQYJKoZIhvcNAQELBQADggEBAAWoThRiOZyhS
IH9I5xT
b4B0I75UwWkKawbfHChnHD/1IEW7SkfzLh1UHAKAp2jMsBoilcq8fgoJF8E4zPJI
lUHvEEfzLuqcCBn3BDoBsAeehQXpHWZyme7nQXbcDfMZEAl38zW9/VySZrWB
Su0s
TLjbJTR+kNMS8iFyIb4dTui7Xm+UadXeCWw3/FfuYbVljWLVsv2a21MwbD5X4
Aga
ozsn18zzFidmdkhl9kJIXsyhqC5IJdpf1xKGvkuYsfP25HhxCyvyHjKHBHnF0vCJ
m48635bLGY7FO3X0F3ZKy3sDtBdb7LErKlIj5uT+vs+WVO2TZMAZp4UHanEEN
vFs

```
Z+g=
-----END CERTIFICATE-----
```

也可直接使用openssl来查看生成的证书，命令如下：

```
openssl x509 -in ./certs/gbasedbt.crt -text -noout
```

例如，查看上一节生成gbasedbt使用的SM2证书，则显示信息如下：

```
[root@localhost gen_8s_cert]# openssl x509 -in ./certs/gbasedbt.crt -text
-noout

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 7 (0x7)
        Signature Algorithm: SM2-with-SM3
        Issuer: C=CN, ST=TIANJIN, O=GBASE, OU=GBASE8S, CN=GBASECA
        Validity
            Not Before: Jul 17 07:53:06 2024 GMT
            Not After : Jul 15 07:53:06 2034 GMT
        Subject: C=CN, ST=TIANJIN, O=GBASE, OU=GBASE8S, CN=gbasedbt
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:77:9c:5c:9d:39:df:6f:0d:69:36:cc:5b:44:7c:
                    c9:b9:b4:e1:2d:e4:e3:75:68:96:f9:c8:74:56:c7:
                    f5:a5:41:5b:17:7f:90:19:c4:f2:77:6d:46:b9:00:
```

30:88:74:3d:9b:9a:4f:82:49:05:a0:83:b3:e8:8d:

f2:bf:89:73:dc

ASN1 OID: SM2

X509v3 extensions:

X509v3 Basic Constraints:

CA:FALSE

X509v3 Key Usage:

Digital Signature, Non Repudiation, Key Encipherment, Data
Encipherment, Key Agreement

X509v3 Subject Key Identifier:

82:DF:81:55:53:64:2D:D4:93:97:C2:43:EF:EF:DB:1A:1F:F0:06:28

X509v3 Authority Key Identifier:

11:59:CC:4D:97:76:EE:4E:14:5A:8C:51:5C:B0:E8:9C:FE:40:D2:3E

Signature Algorithm: SM2-with-SM3

Signature Value:

30:45:02:20:05:39:59:9a:e3:43:bc:5f:56:2b:e5:de:48:82:

5c:8a:39:1f:12:87:ab:bb:ae:df:93:65:14:b0:70:b3:b4:45:

02:21:00:e9:72:8d:81:96:f4:4b:9b:36:1e:fb:49:70:02:7f:

3b:3b:36:b6:02:1f:ce:18:52:20:b1:4a:44:77:01:48:04

12.1.4 8s数据库配置使用SSL/TLS

12.1.4.1. 整体说明

如需使8s支持SSL/TLS协议，需要对如下内容做修改：

归属	文件	说明
ids	onconfig.\${GBASEDBTSERVER} }	需要增加提供ssl的单独vp。示例如下： VPCLASS encrypt , num=3 NETTYPE onsocssl , 3, 50, NET
	sqlhosts.\${GBASEDBTSERVER}	配置ssl所使用的协议及端口，示例如下 gbs onsocssl 192.168.1.100 9088 其中： gbs 8s数据库的实例名为gbs onsocssl 使用的连接协议为ssl/tls 192.168.1.100 网络服务的IP address 9088 网络服务的端口
	sqlhosts.\${GBASEDBTSERVER}. ext	用来指定ssl的配置项，主要服务及客户端所使用的证书链（路径）、 证书校验方式、私钥保护口令等。
	证书目录	用来在本地存放证书（证书文件，根证书和服务端证书，如需本地登录，则也需要有客户端证书） 在本文示例中，证书目录为安装目录下的certs文件夹。
	bin/gbspwdec	命令行工具，用于对证书私钥保护口令进行显示化处理。 如果证书私钥设置了保护口令，8s产品在使用这些证书时需要获得这些证书的私钥保护口令，为了保证安全性，这些证书的私钥保护口令并不会以明文的形式记录在配置文件中。 为此，在sqlhost.\${GBASEDBTSERVER}.ext内填写的证书私钥保护口令这个字段的值是经过加密并经过可显示化处理的。 gbspwdec这个工具就是提供这个证书私钥保护口令的加密并显

		示化处理。 如果GBase 8s产品使用的证书并未设置私钥保护口令，则可以不使用该工具。 例如，证书的私钥保护口令为“11111111”，通过执行如下命令： gbspwdenc 11111111 则获得该证书私钥保护口令的可显示值为： DqOUINa1c2OMcEEbvTG30w== 则，在sqlhosts.\${GBASEDBTSERVER}.ext内填写的私钥保护口令值如下： TLSCertificateKeyFilePasswd=DqOUINa1c2OMcEEbvTG30w==
csdk	sqlhosts.\${GBASEDBTSERVER}	用来连接ssl的服务端的配置，应和服务端内的sqlhost文件一致， 例如： gbs onsocssl 192.168.1.100 9088
	sqlhosts.\${GBASEDBTSERVER}.ext	用来指定ssl的配置项，主要服务及客户端所使用的证书链（路径）、证书校验方式、私钥保护口令等。
	证书目录	用来在本地存放证书（信任根证、客户端的证书）
	bin/gbspwdec	证书私钥加密口令。功能同ids服务端一致。
JDBC	证书目录	用来在本地存放证书（信任根证、客户端的证书）
	keystore	供JDK使用

后续章节将逐项讲述。

12.1.4.2. 服务端设置

12.1.4.2.1. 设置用于加密通讯的资源：onconfig

修改\${GBASEDBTDIR}/etc/onconfig.\${GBASEDBTSERVER}文件，配置GBS参数。

- 使同一实例同时支持普通通讯方式和加密通讯方式：

通过设置DBSERVERALIASES来增加服务的别名，这样可以在sqlhosts配置文件中设置

当前服务支持多种类型的连接，如下示例中实例名`gbs`、`gbs_alias1`都为当前数据库服务实例名，但是在`sqlhosts`配置文件中可以配置为不同的连接类型，这样客户端就可以通过多种通信方式与数据库服务通信：

```
DBSERVERNAME gbs

DBSERVERALIASES gbs_alias1
```

注： 如不需要保留常规通讯`soctcp`模式，则不需要设置别名

● 配置支持通讯加密的VP：

GBS加密和解密操作将由`Encrypt VP`执行。通过`VPCLASS`来配置该类型VP的属性；如果未配置`VPCLASS`，则 `IDS` 会默认启动一个 `Encrypt VP`。`VPCLASS`的配置请参考《GBase8s管理员参考手册》， 示例如下：

```
VPCLASS encrypt,num=3
```

● 配置支持通讯加密的连接方式：

设置 `NETTYPE` 来配置GBS 连接的轮询线程和每个线程的连接数。如果未配置轮询线程,则 `IDS` 将启动一个轮询线程。`NETTYPE`的配置请参考《GBase8s管理员参考手册》， 示例如下：

```
NETTYPE onsocssl,3,50,NET
```

其中，`encrypt.num`和 `NETTYPE`的`onsocssl`数量请根据实际情况调整。

说明：理论上TLS连接比普通的`soctcp`连接要消耗更多的资源，所以单个线程能够处理的连接数也要大幅度下降。

12.1.4.2.2. 设置端口及地址： `sqlhosts`

修改`${GBASEDBTDIR}/etc/sqlhosts.${GBASEDBTSERVER}` 文件，配置GBS参数。
将需要启用传输加密功能的实例的连接类型设置为`onsocssl`；如下述示例中，两个实例数据库服务实例`gbs`、`gbs_alias1`，实例`gbs_alias1`连接类型为`onsoctcp`,为普通的`tcp`通信方式，实例`gbs`的连接类型为`onsocssl`，启用了传输加密方式：

```
gbs_alias1          onsoctcp 192.168.1.100 9089

gbs                  onsocssl 192.168.1.100 9088
```

注： 上述示例中，在192.168.1.100服务器上，8s服务启动两个端口进行监听，在9088端口提供SSL/TLS通讯协议（实例名称为`gbs`），在9089端口则为未加密的`tcp/ip`协议（实

例别名则为gbs_alias1)。

12.1.4.2.3. 设置通讯加密参数: sqlhosts ext

本章的所有操作都需要使用操作系统的gbasedbt用户完成。

需要通过修改\${GBASEBTDIR}/etc/sqlhosts.\${GBASEDBTSERVER}.ext来指定配置参数。

假定8s服务端的安装路径为“/opt/8s_Server”，数据库实例名称为gbs。

如果使用简单传输加密（就是说不使用根证），则配置文件的示例如下：

```
[config]

GBS_TYPE=gbasessl


[gbbs]

server= gbs_server

client=gbs_client


[gbbs_server]

TLSCACertificateFile=/opt/8s_Server/certs/CA.pem

TLSCertificateFile=/opt/8s_Server/certs/server.crt

TLSCertificateKeyFile=/opt/8s_Server/certs/server.key

TLSCertificateKeyFilePasswd=DqOUINa1c2OMcEEbvTG30w==

TLSVerifyCert=verify


[gbbs_client]

TLSCACertificateFile=/opt/8s_Server/certs/CA.crt

TLSCertificateFile=/opt/8s_Server/certs/user1.crt

TLSCertificateKeyFile=/opt/8s_Server/certs/user1.key

TLSCertificateKeyFilePasswd=DqOUINa1c2OMcEEbvTG30w==
```



```
TLSVerifyCert=verify
```

根据配置文件的配置，需要将：

- 1, CAfile(total.pem);
- 2, 服务端证书server.crt和私钥server.key;
- 3, 客户端证书user1.crt和私钥user1.key

保存到\$GBASEDBTDIR/certs中。

在上述ext配置文件中写入gbs_client节的目的是为了为了满足使用在Server端随安装包提供的dbaccess来登录数据库。同时，当csdk和Server端安装在同一目录时，gbs_client节也可以为csdk端登录时指明所用的证书路径。

扩展，如果使用的是双证书（加密和签名使用不同的证书），则上面的证书路径还需要增加加密证书的设置：

```
[gbs_server]
```

```
TLSCACertificateFile=/opt/8s_Server/certs/server/CA.crt
```

```
TLSCertificateFile=/opt/8s_Server/certs/server/server-sign.pem
```

```
TLSCertificateKeyFile=/opt/8s_Server/certs/server/server-sign.key
```

```
TLSEncCertificateFile=/opt/8s_Server/certs/server/server-enc.pem
```

```
TLSEncCertificateKeyFile=/opt/8s_Server/certs/server/server-enc.key
```

```
TLSVerifyCert=verify
```

```
[gbs_client]
```

```
TLSCACertificateFile=/opt/8s_Server/certs/CA.crt
```

```
TLSCertificateFile=/opt/8s_Server/certs/gbasedbt-sign.pem
```

```
TLSCertificateKeyFile=/opt/8s_Server/certs/gbasedbt-sign.key
```

```
TLSEncCertificateFile=/opt/8s_Server/certs/gbasedbt-enc.pem
```

```
TLSEncCertificateKeyFile=/opt/8s_Server/certs/gbasedbt-enc.key
```

```
TLSVerifyCert=verify
```

详细信息请参阅“附录F： \${GBASEDBTSQLHOSTS}.ext的规格”

12.1.4.2.4. 证书目录

该目录存放证书、私钥、根证，包括认证使用的CA证书、Server端证书、客户端证书（如果客户端需要登录）以及对应的私钥等文件。

建议将证书放置在ids安装路径内。证书来源通常是由用户提供，或直接使用附件中提供的证书，或者参阅第一章准备证书，来手工创建证书。

12.1.5 csdk端设置

12.1.5.1. 设置端口及地址：sqlhosts

为了完成csdk向ids的连接，需要将待连接的ids的服务名称、IP、port信息存储在sqlhosts文件内，这个sqlhosts文件的名称不限，但通过csdk登录时必须将环境变量GBASEDBTSQLHOSTS指向到该文件，例如：

```
export GBASEDBTDIR=/opt/csdk  
  
export GBASEDBTSERVER=gbs  
  
export GBASEDBTSQLHOSTS=${GBASEDBTDIR}/etc/sqlhosts.gbs
```

如上所示，该sqlhosts文件的位置就是安装路径下的etc/sqlhost.gbs
此sqlhosts文件的内容可从Server端的sqlhosts文件中获取。

```
gbs      onsocssl 192.168.1.100 9088
```

12.1.5.2. 设置通讯加密参数：sqlhosts ext文件

客户端也需要通过\${GBASEDBTSQLHOSTS}.ext指定GBS配置文件。

假定客户端安装在/opt/csdk中。客户端需要连接的实例名是gbs。用户为gbasedbt，则配置文件示例，如下：

```
[config]  
  
GBS_TYPE=gbasessl  
  
  
[gbs]  
  
client=gbs_client
```

```
[gbs_client]

TLSCACertificateFile=/opt/csdk/certs/CA.crt

TLSCertificateFile=/opt/csdk/certs/gbasedbt.crt

TLSCertificateKeyFile=/opt/csdk/certs/gbasedbt.key

TLSCertificateKeyFilePasswd=DqOUINa1c2OMcEEbvTG30w==

TLSVerifyCert=verify
```

注意：如果使用的证书没有私钥保护口令，则属性值TLSCertificateKeyFilePasswd无需设置。

同ids端一样，如果用户使用的是双证书模式（签名证书+加密证书），则需要增加对加密证书的描述。

```
[config]

GBS_TYPE=gbasessl


[gbs]

client=gbs_client


[gbs_client]

TLSCACertificateFile=/opt/csdk/certs/gbasedbt/CA.crt

TLSCertificateFile=/opt/csdk/certs/gbasedbt/gbasedbt-sign.pem

TLSCertificateKeyFile=/opt/csdk/certs/gbasedbt/gbasedbt-sign.key

TLSEncCertificateFile=/opt/csdk/certs/gbasedbt/gbasedbt-enc.pem

TLSEncCertificateKeyFile=/opt/csdk/certs/gbasedbt/gbasedbt-enc.key

TLSVerifyCert=verify
```

另，如果客户端不提供自己的证书(SSL/TLS通讯时使用服务端的证书完成加解密保护)，则证书文件和私钥文件都可以不设置，此时，客户端向服务端的连接仅使用服务端的

证书。，此时，配置如下所示：

```
[config]

GBS_TYPE=gbasessl


[gsb]

client=gsb_client


[gsb_client]

TLSCACertificateFile=/opt/csdk/certs/gbasedbt/CA.crt

TLSVerifyCert=verify
```

如果客户端不需要验证服务端证书的有效性，则CA证书文件也可以不再设置，此时TLSVerifyCert必须设置为never。ext文件的配置如下：

```
[config]

GBS_TYPE=gbasessl


[gsb]

client=gsb_client


[gsb_client]

TLSVerifyCert=never
```

详细信息请参阅“附录F： \${GBASEDBTSQLHOSTS}.ext的规格”

12.1.5.3. 证书目录

该目录存放证书，包括认证使用的CA证书、客户端证书

建议将证书放置在csdk安装路径内。证书来源通常是由用户提供，或请参阅第一章准备证书，手工创建证书。

12. 1. 6 JDBC端设置

12.1.6.1. 生成JAVA所需的证书库

在前一节生成证书后，可通过JDK/JRE（1.8版本之上）或者GBase 8s安装包内提供的keytool工具，将开源工具（openssl或铜锁）生成的证书文件（crt文件）转换为java端所需的证书格式。

以生成user1用户的证书库为例，具体操作如下：

```
keytool \  
  
    -importcert \  
  
    -keypass "11111111" -storepass "11111111" \  
  
    -noprompt \  
  
    -file "./certs/user1.crt" \  
  
    -keystore "./certs/user1.keystore"
```

参数说明如下：

参数	解释
-keypass	输入证书文件的私钥保护口令， 本例中为：11111111
-storepass	生成的证书池的私钥保护口令， 本例中为：11111111
-file	待转换的证书文件
-keystore	输出的java能够识别的证书

12.1.6.2. 配置并启用JDBC SSL

在java程序中需要通过如下步骤开启ssl，红色部分需要根据实际情况来设置：

- 1) 设置信任凭证及密码，有两种方式
- 方式1：通过设置如下属性

```
/* System properties for keystore */  
  
System.setProperty("javax.net.ssl.trustStore", "/绝对路径/.keystore");
```

```
System.setProperty("javax.net.ssl.trustStorePassword", "证书文件密码");
```

方式2：通过在java启动命令中使用-D参数

```
-Djavax.net.ssl.trustStore=/绝对路径/.keystore  
-Djavax.net.ssl.trustStorePassword=证书文件密码
```

2) JDBC开启ssl传输加密，根据使用方式也有两种

方式1：通过DataSource代码开启

```
/* Instantiate Gbasedbt connection pooled data source */  
  
IfxConnectionPoolDataSource cds = new IfxConnectionPoolDataSource();  
  
cds.setIfxSSLCONNECTION("true"); //启用ssl加密
```

方式2：jdbc url设置参数sslConnection=true 开启

12.1.6.3. 支持TLS 1.3

JDBC对tls1.3的支持依赖JDK的版本。从JDK11开始，正式支持tls1.3,故只需升级到jdk11及以上的任意版本即可。

当然jdk8后续的补丁版本也支持tls1.3，但需要注意如下：

- Oraclejdk 8u261及以上版本支持tls1.3
- Openjdk 8u302及以上版本支持tls1.3
- 使用jdk8的以上版本时默认使用的仍是tls1.2,如需使用tls1.3,需要通过
“-Djdk.tls.client.protocols=TLSv1.3”开启。

12.1.6.4. jdbc验证用例

本节提供了一个验证jdbc传输加密功能的示例程序，连接数据库服务并执行一条简单查询以验证连接成功，用户根据实际情况修改示例程序中的红色部分，然后按照步骤编译运行即可。

```
public static final String url="jdbc:gbasedbt-sqli:"           // 驱动名称  
  
    + "///192.168.0.101:25625/sysmaster:"                   // IP 端口 库名  
  
    + "GBASEDBTSERVER=ol_gbasedbt1210_1;"                  // 实例名称  
  
    + "SQLMODE=GBase;"                                       // sql 模式or Oracle
```

```
+ "SSLConnection=true;" // 使用 SSL/TLS

+ "DB_LOCALE=en_US.819"; // 字符集

public static final String username = "gbasedbt"; // 用户名

public static final String password = "111111"; // 用户口令

public static final String certpath = "/opt/gbase_3513/certs/"; // 用户证书路径

static {

    Security.removeProvider("SunEC"); // 移除默认的JCE provider (使用bc)

    System.setProperty("javax.net.ssl.keyStore", certpath + "gbasedbt.jks"); // 设置用户证书

    System.setProperty("javax.net.ssl.keyStorePassword", "11111111"); // 用户证书私钥保
护口令

    System.setProperty("javax.net.ssl.trustStore", certpath + "CA.jks"); // CA证书

    System.setProperty("javax.net.ssl.trustStorePassword", "11111111"); // CA证书私钥保护口
令

//    System.setProperty("javax.net.debug", "all");

    try {

        Class.forName("com.gbasedbt.jdbc.Driver");

    } catch (ClassNotFoundException e) {

        e.printStackTrace();

    }

}

public static void main(String[] args) throws Exception {

    // 建立连接

    System.out.println(" < Step 1 > : connect to server: " );
```

```
System.out.println("          connect to: " + url);

Connection connection = DriverManager.getConnection (url, username, password);

System.out.println("          The connection is successful." );

// 创建一个sql statement句柄

System.out.println(" < Step 2 > : Try to execute sql: select 1 from dual" );

Statement statement = connection.createStatement();

// 执行一条简单sql

ResultSet resultset = statement.executeQuery("select 1 from dual;");

System.out.println("          sql execute is successful, result  is :");

// 输出执行结果

while(resultset.next()) {

    System.out.println("          " + resultset.getString(1));

}

// 断开连接

System.out.println(" < Step 3 > : close connect" );

if(connection != null) {

    connection.close();

}

// 完成测试

System.out.println(" < Finished > :");
```



```
System.out.println("    Congratulations, JDBC test successful.");  
}
```

编译并运行：

```
javac ./TestJDBC.java  
java -cp ../gbasedbtjdbc_3.3.0_2P20220402_428c64.jar TestJDBC
```

其中，gbasedbtjdbc_3.3.0_2P20220402_428c64.jar请替换为GBase 8s提供的对应版本的jdbc驱动。

如连接成功将显示ssl连接成功。

```
[root@localhost src]# java -cp ../gbasedbtjdbc_3.3.0_2P20220402_428c64.jar  
TestJDBC  
  
    < Step 1 > : connect to server:  
  
jdbc:gbasedbt-sqli://192.168.0.101:25625/sysmaster:GBASEDBTSERVER=ol_g  
basedbt1210_1;SQLMODE=GBase;SSLConnection=true;DB_LOCALE=en_US.81  
9;  
  
        The connection is successful.  
  
    < Step 2 > : Try to execute sql: select 1 from dual  
  
        sql execute is successful, result  is :  
  
        1  
  
    < Step 3 > : close connect  
  
    < Finished > :  
  
        Congratulations, JDBC test successful.
```

如连接成功，会显示如上提示信息。

12.1.7 图形界面的设置

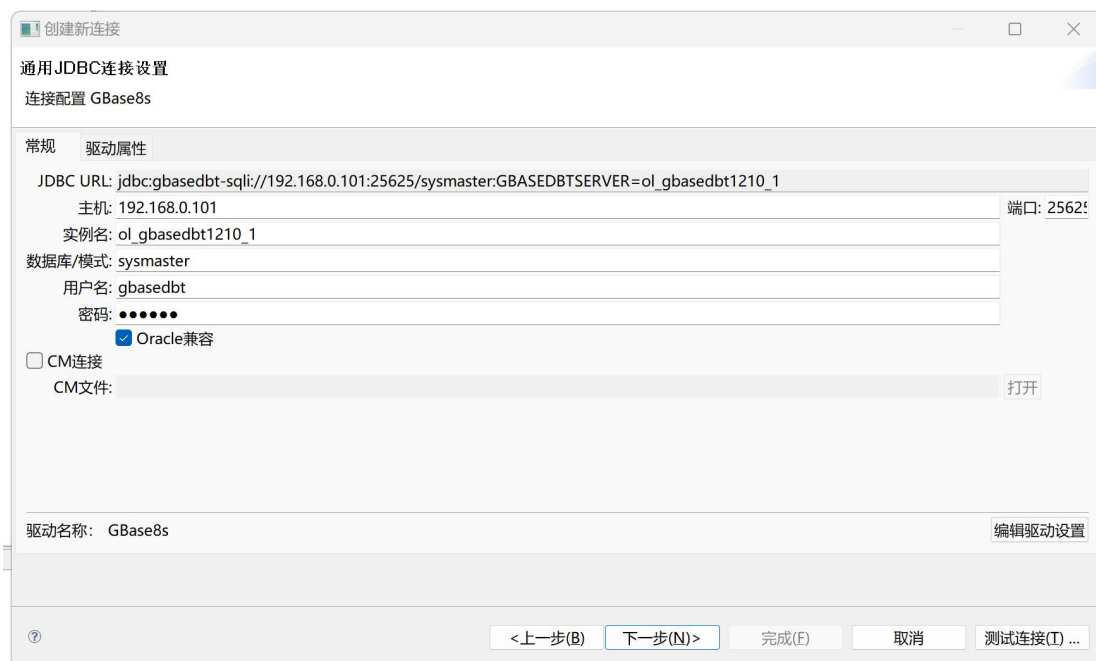
使用图形工具登录GBase 8s时，需要设定登录用户所使用的证书（以及信任的CA）才能使用SSL/TLS通讯。

以下示例以GBase 8s的GBase Data Studio工具为例，讲述如何通过配置使图形化工具支持SSL/TLS连接。

12.1.7.1. 新建连接

具体设置如下：

1、 点击菜单 “数据库” -> “新建连接”



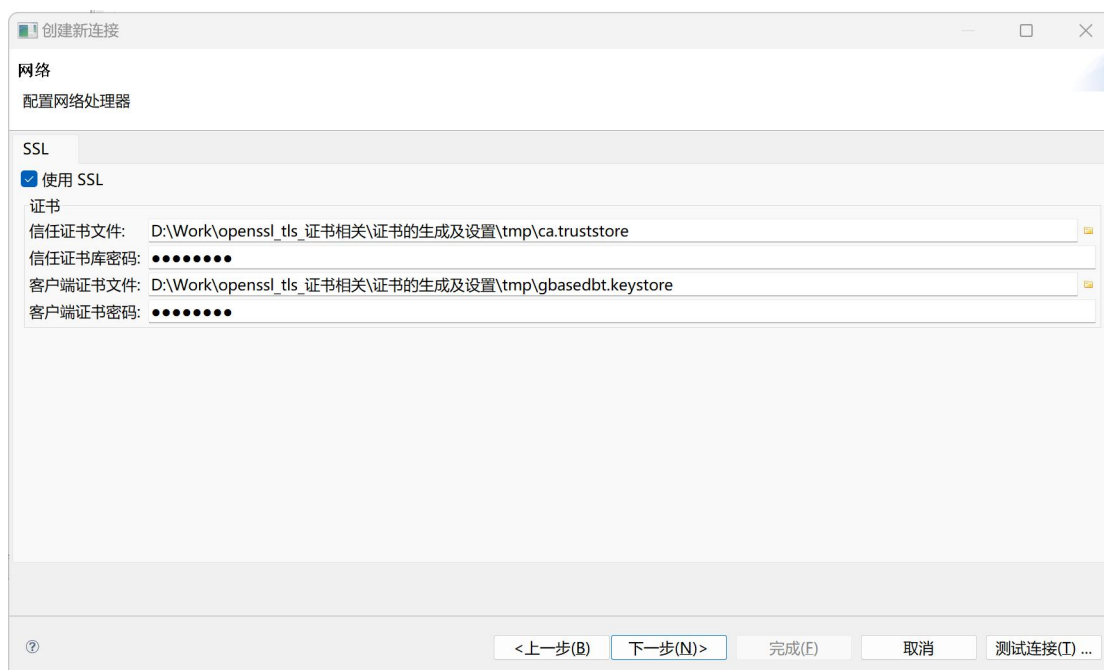
根据数据库实际信息填写主机、实例名、数据库、用户名、密码等信息。

如不确认，请参阅前一节“设置端口及地址：sqlhosts”的相关内容。

2、 点击下一步，勾选“使用SSL”



3、 填写信任证书文件（CA）、信任证书库密码、客户端证书文件、客户端证书密码：



注： 相应证书文件的生成可参照前一节“生成JAVA所需的证书库”的内容。

4、 点击界面右下角的测试连接按钮，弹出如下窗口，说明配置成功



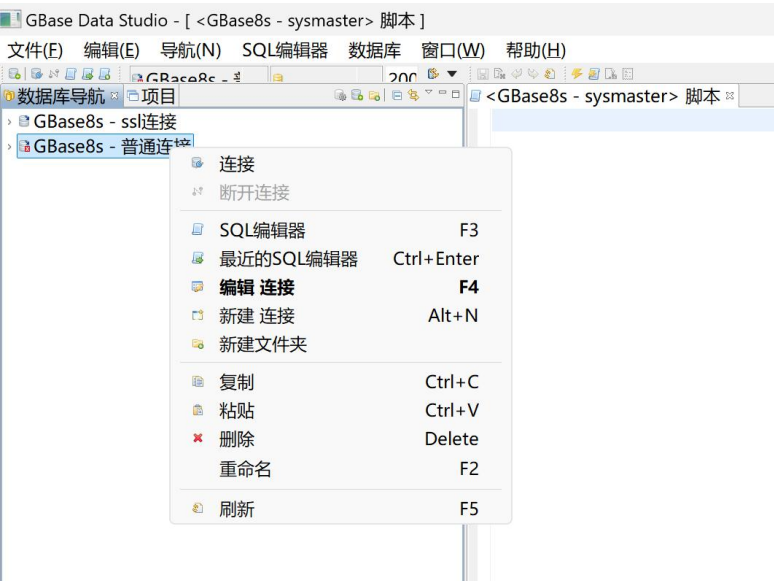
注：由于SSL/TLS算法相对常规tcp/ip连接来说增加了握手和加密等操作步骤。主要是在建立连接时要比不使用SSL/TLS协议的连接建立会慢一些,对服务端资源的占用也较大。SSL/TLS连接建立后，则差异将会降低。

5、 点击下一步，完成其他配置后，点击完成。

12.1.7.2. 修改已有连接

当希望将GDS已保存的非加密连接修改为SSL/TLS加密连接，则需要执行如下操作：

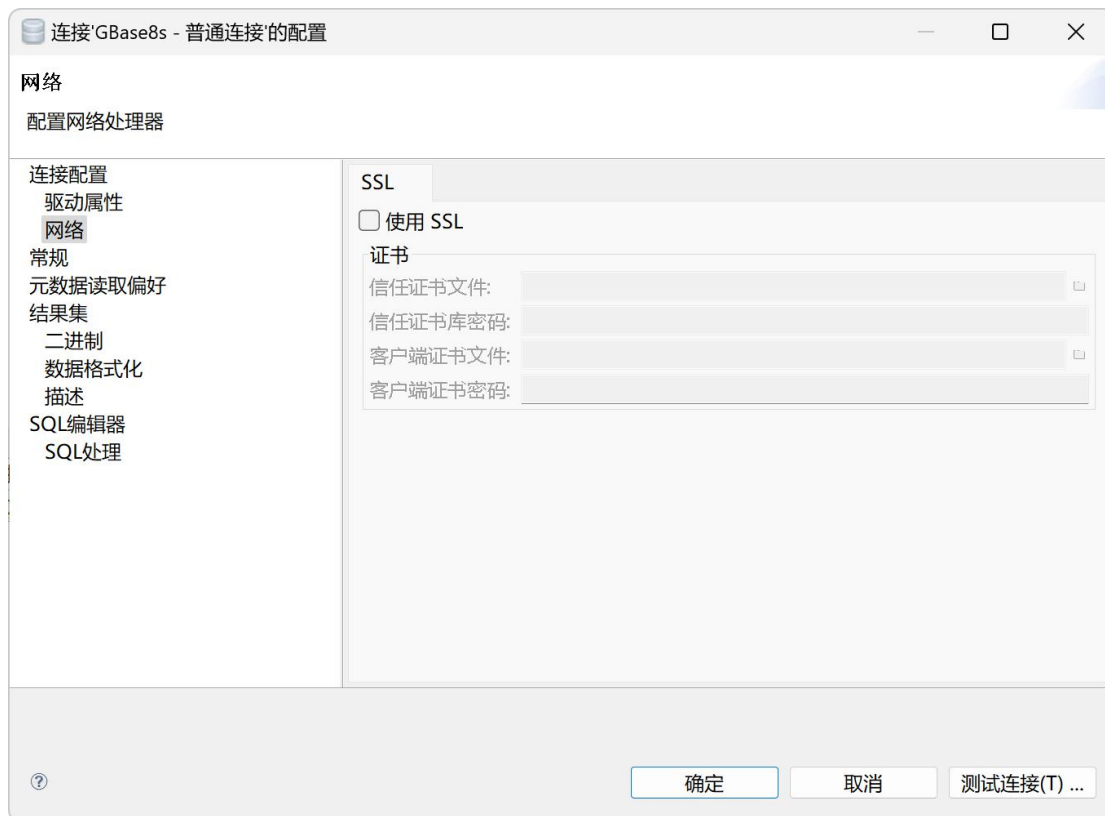
- 1、 打开GDS工具，鼠标选中待修改的连接，按“F4”键，或鼠标右键，在弹出的菜单中选择“编辑连接”



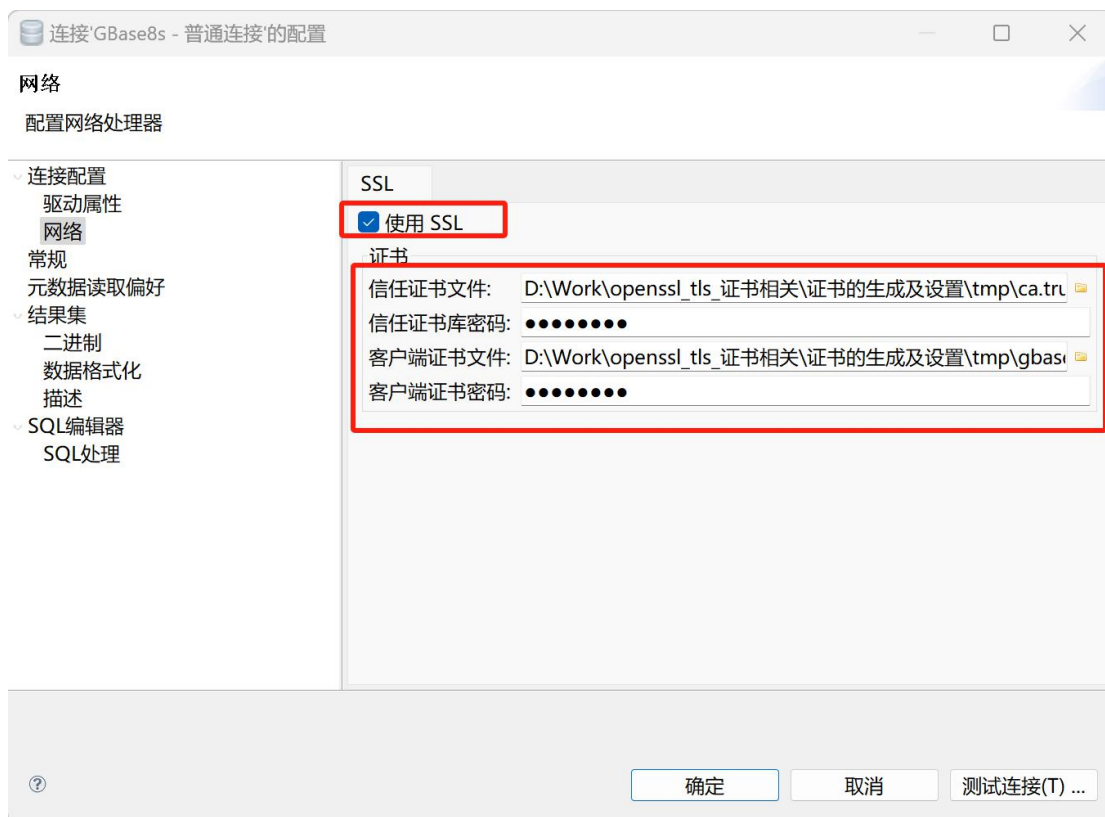
注：在本示例中，GDS内已建立了两个连接“GBase8s -ssl连接”和“GBase8s -普通连

接”，其中“GBase8s -普通连接”为本次待修改的连接。

2、 在弹出的对话框中，选择“网络”



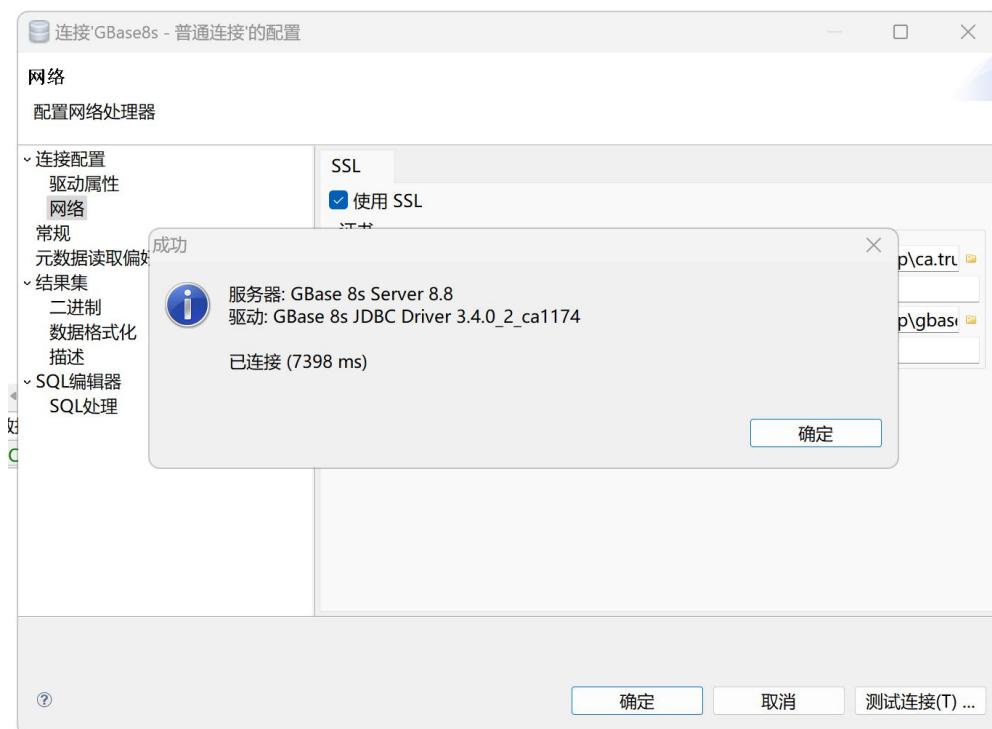
3、 勾选“使用SSL”，在激活SSL配置框后，输入“信任证书文件”、“信任证书库密码”、“客户端证书文件”、“客户端证书密码”。



注：相应证书文件的生成可参照前一节“生成JAVA所需的证书库”的内容。

如SSL/TLS通讯仅使用Server端证书，则“客户端证书文件”、“客户端证书密码”可填空。

4、 点击界面右下角的测试连接按钮，弹出如下窗口，说明配置成功



5、 点击“确定”，关闭弹出的成功提示框，再次点击“确定”完成本次配置。

12. 1. 8 高可用集群使用SSL

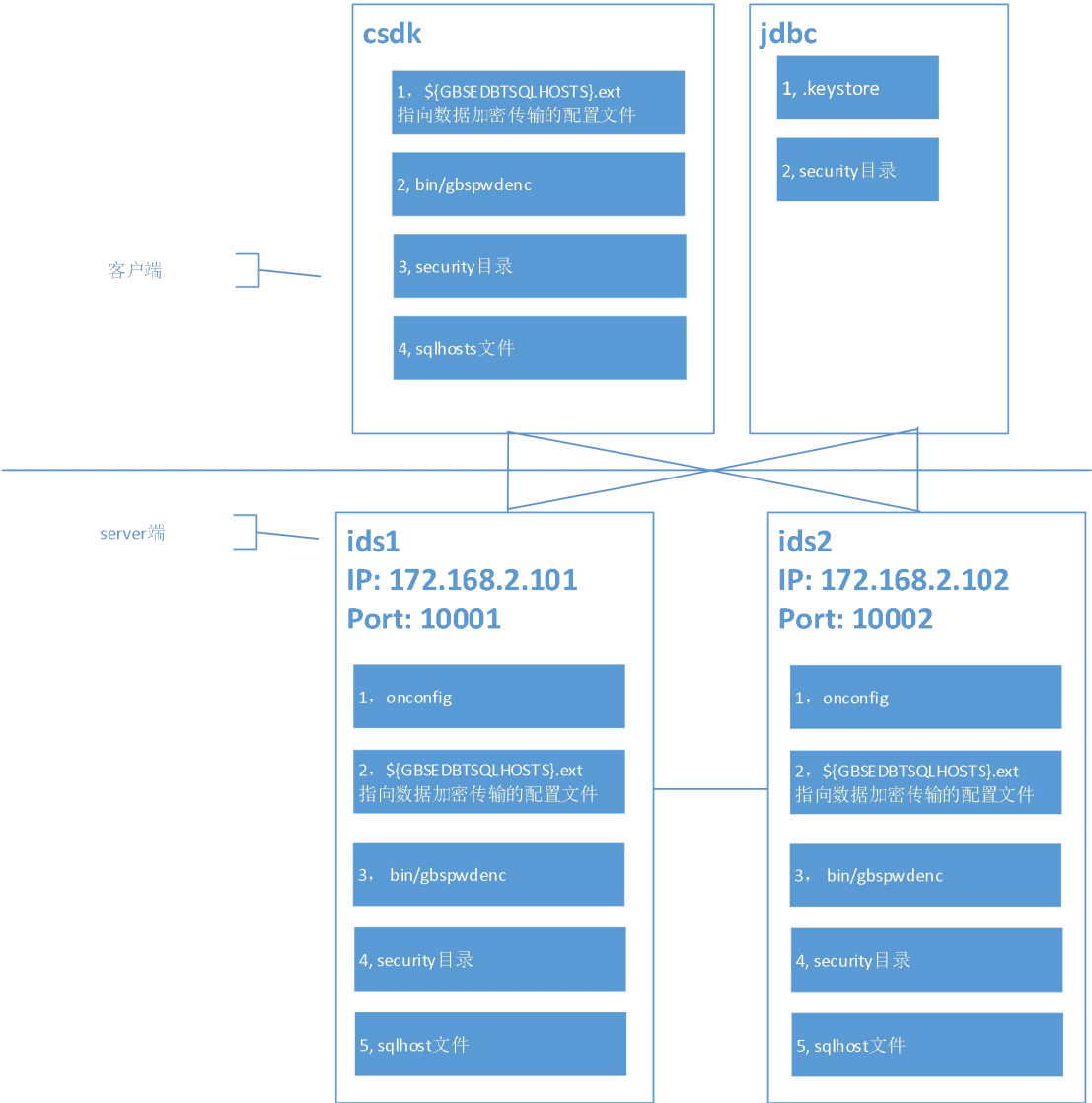
高可用集群包含：SDS, HDR, RSS三种情况。对于传输加密而言这三种的配置方法是相同的。

为了监视和维护集群的客户机连接，以及将客户机连接请求定向到连接单元中合适的服务器上，引入了连接管理器。

本章将分别介绍基于不使用连接管理器的集群和使用连接管理器的集群如何配置安全通讯。本文不介绍如何搭建高可用集群。

本章用ids1代表主节点，ids2代表从节点。

12.1.8.1. 不使用连接管理器的集群



针对每个server端和客户端，需要部署的内容都和单机安全部署相同，相应的介绍请参阅：2 单机安全通讯部署。

12.1.8.1.1. 搭建集群

在搭建使用SSL/TLS的高可用集群之前，首先需要安装部署一个使用非加密连接的集群。
具体请参见：Administrator's Guide

在搭建完非加密高可用集群后，需要正常关闭集群，之后进行SSL/TLS配置。

12.1.8.1.2. 配置数据库服务端

12.1.8.1.2.1. 准备服务端证书文件

准备服务端证书文件的方法和单机模式完全相同。
多个ids需要配置相同的根证文件。

12.1.8.1.2.2. 启用加密

启用加密的方法和单机模式相同。
关于sqlhosts文件，需要同时包含主从实例，举例如下。

#ids组和各个ids				
db_group	group	-	-	i=10,c=1
ids1	onsocssl	172.168.2.101	10001	g=db_group
ids2	onsocssl	172.168.2.102	10002	g=db_group

12.1.8.1.2.3. 撰写配置文件

对于ids1而言，配置文件的内容如下：

```
[config]
GBS_TYPE=gbasessl

[ids1]
server=ids1_server
client=ids1_client

[ids1_server]
TLSCACertificateFile= <full path>
```

```
TLSCertificateFile= <full_path>
TLSCertificateKeyFile= <full_path>
TLSCertificateKeyFilePasswd= <password>
TLSVerifyCert={verify | try | never}
```

```
[ids1_client]
```

```
TLSCACertificateFile= <full path>
TLSCertificateFile= <full_path>
TLSCertificateKeyFile= <full_path>
TLSCertificateKeyFilePasswd= <password>
TLSVerifyCert={verify | try | never}
```

```
[ids2]
```

```
client=ids2_client
```

```
[ids2_client]
```

```
TLSCACertificateFile= <full path>
TLSCertificateFile= <full_path>
TLSCertificateKeyFile= <full_path>
TLSCertificateKeyFilePasswd= <password>
TLSVerifyCert={verify | try | never}
```

对于ids2而言，配置文件的内容如下：

```
[config]
```

```
GBS_TYPE=gbasessl
```

[ids2]

server=ids2_server

client=ids2_client

[ids2_server]

TLSCACertificateFile= <full path>

TLSCertificateFile= <full_path>

TLSCertificateKeyFile= <full_path>

TLSCertificateKeyFilePasswd= <password>

TLSVerifyCert={verify | try | never}

[ids2_client]

TLSCACertificateFile= <full path>

TLSCertificateFile= <full_path>

TLSCertificateKeyFile= <full_path>

TLSCertificateKeyFilePasswd= <password>

TLSVerifyCert={verify | try | never}

[ids1]

client=ids1_client

[ids1_client]

TLSCACertificateFile= <full path>

TLSCertificateFile= <full_path>

```
TLSCertificateKeyFile= <full_path>  
  
TLSCertificateKeyFilePasswd= <password>  
  
TLSVerifyCert={verify | try | never}
```

12.1.8.1.2.4. 启动服务器。

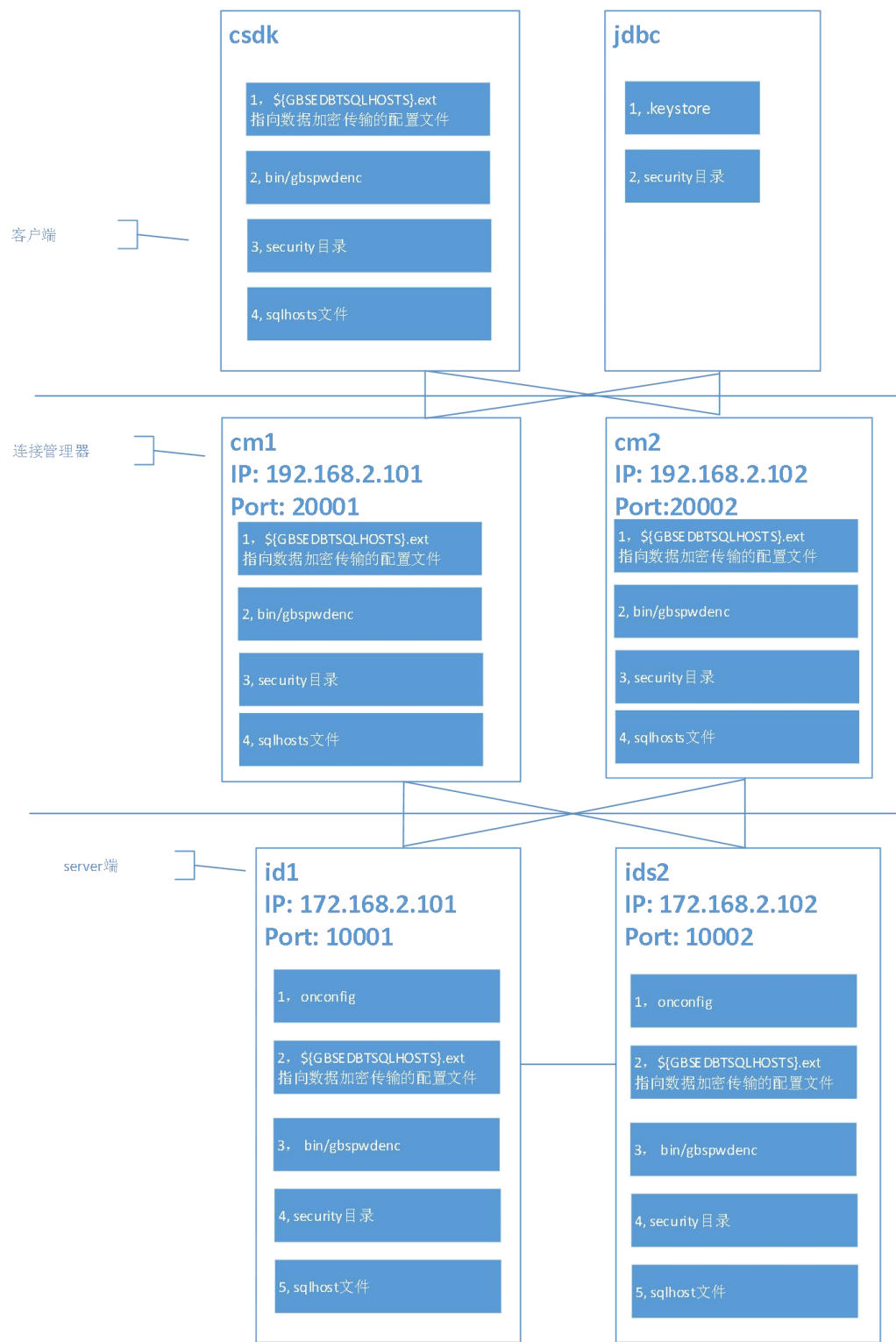
完成配置后，需要重新启动集群以使配置生效。

12.1.8.1.3. 配置数据库客户端。

无论CSDK还是JDBC，配置都和单机的是相同的。

12.1.8.2. 使用连接管理器的集群

使用连接管理器CM的集群架构如下：



12.1.8.2.1. 搭建集群

同不使用连接管理器的集群相同，在搭建使用SSL/TLS的高可用集群之前，首先需要安装部署一个使用非加密连接的集群。具体请参见：Administrator's Guide

在搭建完非加密高可用集群后，需要正常关闭集群，之后进行SSL/TLS配置。

12.1.8.2.2. 配置数据库服务端

ids端的配置和不使用连接管理器的集群的完全相同。

12.1.8.2.2.1. 准备服务端证书文件

准备服务端证书文件的方法和单机的完全相同。

多个ids需要配置相同的根证书。

12.1.8.2.2.2. 启用加密

启用加密的方法和单机模式相同。

关于sqlhosts文件，需要同时包含主从实例，举例如下。

#ids组和各个ids				
db_group	group	-	-	i=10,c=1
ids1	onsocssl	172.168.2.101	10001	g=db_group
ids2	onsocssl	172.168.2.102	10002	g=db_group

12.1.8.2.2.3. 撰写配置文件

配置文件的内容和不使用连接管理器的配置文件完全相同。

12.1.8.2.2.4. 启动服务器。

完成配置后，需要重新启动集群以使配置生效。

12.1.8.3. 配置连接管理器端

CM端本身是通过csdk安装配置的，在访问server端的时候，它是客户端；在给客户端提供服务的时候，它是server端。

- 1， security目录，必须和ids端使用同一个根证。
- 2， sqlhost文件。连接管理器的sqlhost文件有很多种写法。这里只介绍普通soctcp连接和安全通讯socssl连接的差异。就是将连接类型修改为socssl。根据上述的部署图，举例如下：

#ids组和各个ids				
db_group	group	-	-	i=10,c=1
ids1	onsocssl	172.168.2.101	10001	g=db_group

ids2	onsocssl	172.168.2.102	10002	g=db_group
#连接管理器组和各个连接管理器				
cm_g	group	-	-	i=12,c=1
cm1	onsocssl	192.168.2.101	20001	g=cm_g
cm2	onsocssl	192.168.2.101	20002	g=cm_g

3，撰写配置文件：

```
[config]

GBS_TYPE=gbasessl


[ids1]

client=ids1_client


[ids1_client]

TLSCACertificateFile= <full path>
TLSCertificateFile= <full_path>
TLSCertificateKeyFile= <full_path>
TLSCertificateKeyFilePasswd= <password>
TLSVerifyCert={verify | try | never}


[ids2]

client=ids2_client


[ids2_client]

TLSCACertificateFile= <full path>
```

```
TLSCertificateFile= <full_path>  
TLSCertificateKeyFile= <full_path>  
TLSCertificateKeyFilePasswd= <password>  
TLSVerifyCert={verify | try | never}
```

```
[cm1]
```

```
server=cm1_server
```

```
client=cm1_client
```

```
[cm1_server]
```

```
TLSCACertificateFile= <full path>
```

```
TLSCertificateFile= <full_path>
```

```
TLSCertificateKeyFile= <full_path>
```

```
TLSCertificateKeyFilePasswd= <password>
```

```
TLSVerifyCert={verify | try | never}
```

```
[cm1_client]
```

```
TLSCACertificateFile= <full path>
```

```
TLSCertificateFile= <full_path>
```

```
TLSCertificateKeyFile= <full_path>
```

```
TLSCertificateKeyFilePasswd= <password>
```

```
TLSVerifyCert={verify | try | never}
```

12.1.8.4. 配置数据库客户端

无论CSDK还是JDBC，里面部署的文件都和单机的是相同的。

使用和不使用连接管理器的差异是连接的目标不同，具体如下：

不使用连接管理器的集群	使用连接管理器的集群
连接的目标是ids1, ids2。	连接的目标是cm1, cm2

12.1.9 对国密算法的支持

在前文中，已经描述了国密证书的生成。详细请参阅“生成国密算法证书”。在GBase 8s服务端以及CSDK端（包括odbc、esqlc、GCI），使用国密算法证书和使用其他算法的证书没有区别。

在jdbc端（也包括使用jdbc连接的图形化工具例如GDS等），如果使用国密算法的证书，受限于JDK原生安全算法的限制，需要做一些操作才能正确的生成java能够识别的证书格式。下文将详细讲述这些区别和对应的操作。

注，为了支持国密算法，请将JDK或JRE升级到如下版本或更新的版本：

- Oraclejdk 8u261及以上版本
- Openjdk 8u302及以上版本

下文中的示例以Oracle JDK 1.8.0_401作为演示。

将 SM2 加密标准的 JCE(Java Cryptography Extension)加入到JDK中涉及到几个步骤。SM2 是中国国家密码管理局制定的一种椭圆曲线密码算法，通常需要第三方的 JCE 提供者来支持，如 Bouncy Castle或GMSSL。

在本文中，以Bouncy Castle来进行示例。

12.1.9.1. 安装支持国密算法的JCE: Bouncy Castle

Bouncy Castle 提供了各种密码学算法的实现，包括对称加密算法（如 AES、DES、RC4）、非对称加密算法（如 RSA、DSA、ECC）、哈希算法（如 MD5、SHA-1、SHA-256）、消息认证码（MAC）算法、数字签名算法以及各种密钥交换协议等。此外，Bouncy Castle 还支持密码学协议的实现，如 SSL/TLS、S/MIME、OpenPGP 等。

官网地址：

<https://www.bouncycastle.org/>

以本示例1.78.1版本为例，下载地址如下：

<https://downloads.bouncycastle.org/java/bcprov-jdk15to18-1.78.1.jar>

或直接使用附件：



bcprov-jdk15to18-1.78.1.jar

12.1.9.1.1. 动态安装的方式

这种方式是在工程中直接使用 Bouncy Castle，对环境的依赖较小。

将 bcprov-jdk15on-158.jar 导入工程

在使用之前动态加入BouncyCastle Provider

示例代码如下：

```
public class hello {  
  
    //动态使用provider  
  
    static {  
  
        Security.addProvider(new BouncyCastleProvider());  
  
    }  
  
    public static void main(String args[]) {  
  
        System.out.println("请输入待加密数据：");  
  
        Scanner scan = new Scanner(System.in);  
  
        while (scan.hasNext()) {  
  
            try {  
  
                String date = scan.nextLine();  
  
                KeyGenerator kg =KeyGenerator.getInstance("AES");  
  
                kg.init(256);  
  
                SecretKey secretKey = kg.generateKey();  
  
                Cipher cipher =  
Cipher.getInstance("AES/CBC/PKCS7Padding", "BC");  
  
                cipher.init(Cipher.ENCRYPT_MODE, secretKey);  
  
                byte[] enResult = cipher.doFinal(date.getBytes());  
  
                System.out.println("加密数据：");  
  
                String strEnResult = Base64.toBase64String(enResult);  
  
                System.out.println(strEnResult);  
  
            }  
  
        }  
  
    }  
  
}
```

```
        catch (NoSuchAlgorithmException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        catch (NoSuchProviderException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        catch (NoSuchPaddingException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        catch (InvalidKeyException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        catch (IllegalBlockSizeException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        catch (BadPaddingException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
    }  
}
```

```
}  
  
}
```

12.1.9.1.2. 静态配置的方式

静态配置方式是本地 Java 环境中添加 Bouncy Castle 支持，需要使用到的 jar 包是 bcprov-jdk15to18-1.78.1.jar。

把 bcprov-jdk15to18-1.78.1.jar 添加到java 的JRE目录中。

windows示例：

拷贝位置→ C:\Program Files\Java\jre1.8.0_401\lib\ext

Linux下示例：

拷贝位置→ /home/jdk/jdk1.8.0_421/jre/lib/ext

修改security配置文件java.security，为了更好的体验，最好将sdk和jre中security文件都进行修改。

JRE中security文件位置：

\jre\lib\security\java.security

在 security 文件中找到以下部分：

```
#  
  
# List of providers and their preference orders (see above):  
  
#  
security.provider.1=sun.security.provider.Sun  
security.provider.2=sun.security.rsa.SunRsaSign  
security.provider.3=sun.security.ec.SunEC  
security.provider.4=com.sun.net.ssl.internal.ssl.Provider  
security.provider.5=com.sun.crypto.provider.SunJCE  
security.provider.6=sun.security.jgss.SunProvider  
security.provider.7=com.sun.security.sasl.Provider
```

```
security.provider.8=org.jcp.xml.dsig.internal.dom.XMLDSigRI
security.provider.9=sun.security.smartcardio.SunPCSC

#

# Sun Provider SecureRandom seed source.

#
```

为了使得BouncyCastle提供的安全算法能够被首先使用，在provider列表前面添加如下这一行，并修改剩余provider的序号：

```
#

# List of providers and their preference orders (see above):

#

security.provider.1=org.bouncycastle.jce.provider.BouncyCastleProvider
security.provider..2=sun.security.provider.Sun
security.provider..3=sun.security.rsa.SunRsaSign
security.provider..4=sun.security.ec.SunEC
security.provider..5=com.sun.net.ssl.internal.ssl.Provider
security.provider..6=com.sun.crypto.provider.SunJCE
security.provider..7=sun.security.jgss.SunProvider
security.provider..8=com.sun.security.sasl.Provider
security.provider..9=org.jcp.xml.dsig.internal.dom.XMLDSigRI
security.provider..10=sun.security.smartcardio.SunPCSC
ecurity.provider..11=security.provider.10=sun.security.smartcardio.SunPCSC

#

# Sun Provider SecureRandom seed source.
```

#

12.1.9.2. java key store的生成

java使用自己的证书库来提供安全连接，并提供了keytool工具来实现证书的生成和转换。

例如如下的语句就是将OpenSSL生成的RSA证书转换为java的证书文件：

```
keytool \  
    -importcert \  
    -keypass "11111111" -storepass "11111111" \  
    -noprompt \  
    -file "./certs/server.crt" \  
    -keystore "./certs/server.keystore"
```

而在使用国密算法证书时，因国密证书算法以及格式的差异，并不能使用keytool工具完成上述的操作。本文档通过使用BouncyCastle算法库，来实现这一证书转换操作。

12.1.9.2.1. 代码示例

示例代码如下：

```
public static void PKCS12ToJKS(String pathSrc, String passwd, String pathDst) throws  
IOException, CertificateException, NoSuchAlgorithmException, NoSuchProviderException,  
KeyStoreException, UnrecoverableKeyException {  
  
    char []passwd_c = passwd.toCharArray();  
  
    System.out.println("    < Parameters List > ");  
    System.out.println("        source cert file ----- '" + pathSrc + "'");  
    System.out.println("        private key passwd ----- '" + passwd + "'");  
    System.out.println("        dest cert file ----- '" + pathDst + "'");
```

```
// removed default JCE

Security.removeProvider("SunEC");

// 读取给定的pcsk12格式的证书

System.out.println(" < Step 1 > : load pkcs12 cert" );

Security.addProvider(new BouncyCastleProvider());

FileInputStream fs = new FileInputStream(pathSrc);

KeyStore store = KeyStore.getInstance("PKCS12", "BC");

store.load(fs, passwd_c );

fs.close();


//生成一个空的jks , 以待输出

System.out.println(" < Step 2 > : general key store" );

KeyStore outputKeyStore = KeyStore.getInstance("JKS");

outputKeyStore.load(null, passwd_c);


//遍历证书信息 , 写入到待输出的jks中

System.out.println(" < Step 3 > : load cert element" );

Enumeration<String> enumas = store.aliases();

while (enumas.hasMoreElements()) {

    String keyAlias = (String) enumas.nextElement();

    System.out.println("          : alias=[" + keyAlias + "]);

    if (store.isKeyEntry(keyAlias)) {

        Key key = store.getKey(keyAlias, passwd_c);    //获取私钥

        Certificate[] certChain = store.getCertificateChain(keyAlias);
```



```
        outputKeyStore.setKeyEntry(keyAlias, key, passwd_c, certChain);
    }
}

//将新的jks写入到指定文件内

System.out.println("    < Step 4 > : general jks file" );

FileOutputStream out = new FileOutputStream(pathDst);

outputKeyStore.store(out, passwd_c);

out.close();


System.out.println("    < Finished > : ");

System.out.println("        Congratulations, JKS [" + pathDst + "] has been successfully
generated.");
}
```

该示例程序读取给定证书文件和私钥保护口令，并输出到指定证书路径。

详细源码内容请参阅附件。

12.1.9.2.2. 代码使用

本示例源码要求jdk/jre已安装支持国密算法的JCE（请参阅前一章节）。在本示例中，国密算法支持库Bouncy Castle安装到了JRE 1.8.0_421内，运行环境为Centos。

示例代码文件为GenJKS.java，存储在当前路径的src文件夹下，当前路径下的“cert_sm2”文件夹内存储着待转换的证书文件。

```
[root@localhost testsm2]# java -version
```

```
java version "1.8.0_421"
```

```
Java(TM) SE Runtime Environment (build 1.8.0_421-b09)
```

```
Java HotSpot(TM) 64-Bit Server VM (build 25.421-b09, mixed mode)
```

```
[root@localhost testsm2]# javac ./src/GenJKS.java

[root@localhost testsm2]# java -cp ./src GenJKS ./certs_sm2/CA.pfx

11111111 ./certs_sm2/CA.jks

< Parameters List >

source cert file ----- './certs_sm2/CA.pfx'

private key passwd ----- '11111111'

dest cert file ----- './certs_sm2/CA.jks'

< Step 1 > : load pkcs12 cert

< Step 2 > : general key store

< Step 3 > : load cert element

: alias=[95844bab16003e9875ee3f2dadd8f38ac7af99bc]

< Step 4 > : general jks file

< Finished > :

Congratulations, JKS [./certs_sm2/CA.jks] has been successfully generated.
```

关键点:

- 1、jdk版本为1.8.0_421或更高;
- 2、支持国密算法的JCE文件“bcprov-jdk15to18-1.78.1.jar”（或其他版本）以拷贝到了JRE内的“./lib/ext/bcprov-jdk15to18-1.78.1.jar”文件夹下;
- 3、修改JRE目录下的“./lib/security/java.security”文件，将“security.provider.1=org.bouncycastle.jce.provider.BouncyCastleProvider”放置在provider列表的第一行，并将之后其他的provider的数字顺序增加;
- 4、通过which命令确保使用的java和javac是安装好支持国密JCE的JDK和JRE。

12.1.10 验证SSL/TLS

可采用抓包分析的方法来验证通讯使用了SSL/TLS，并确认使用的版本。

在Server端启动tcpdump，捕获客户端登录server端的网络包并存储为文件，通过windows下的wireshark工具即可清晰的查看登录使用的ssl连接版本。

1、抓取网络包

抓包示例：以端口25625为例：

```
tcpdump -i ens33 port 25625 -w /home/dump1.pcap
```

参数释义如下：

参数	说明
-i ens 33	在本示例中使用的虚拟网卡，设备名为ens33 如果常规单网卡下，该参数不指定时使用的默认网卡为：eth0
port 25625	指定8s server的服务端口
-w /home/lx/dump1.pcap	输出文件

2、建立SSL/TLS连接

在通过tcpdump启动抓包后，使用客户端（gds工具或其他应用端）登录8s数据库。

- 请确认tcpdump启动在登录操作之前；
- 请确认确认登录成功。

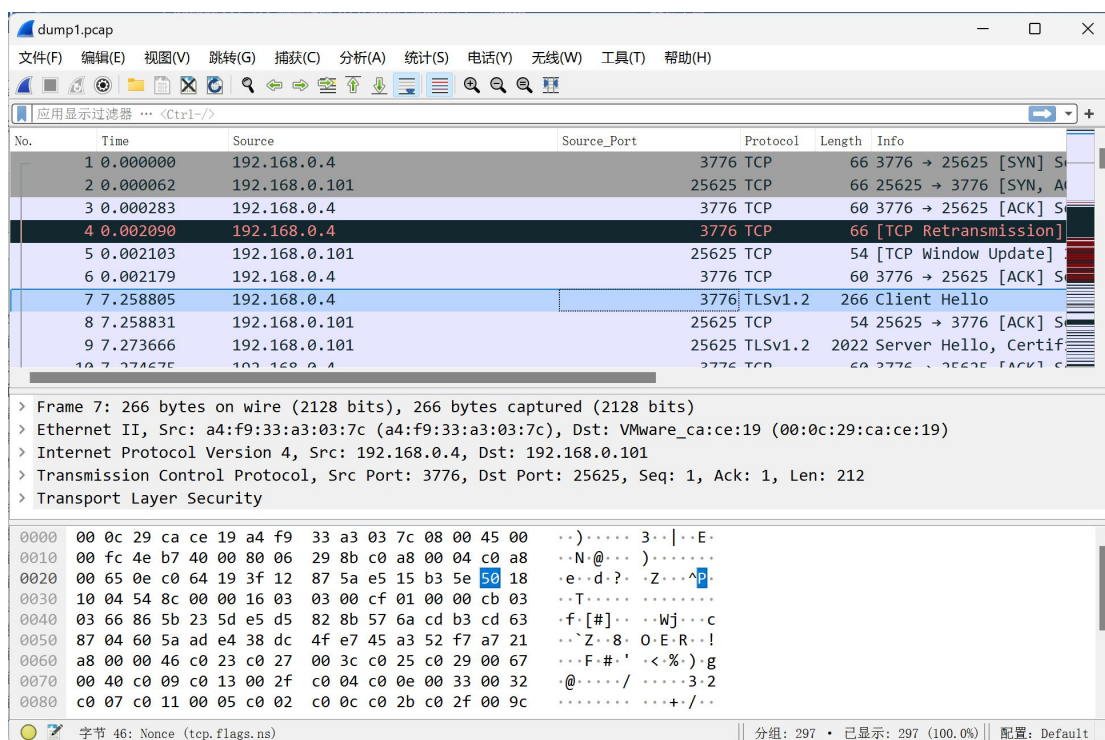
3、关闭抓包并生成网络包文件

通过 **ctrl+c**的方式关闭tcpdump的抓包操作。关闭成功后tcpdump会输出显示本次抓包操作最终抓到了多少个网络包。

```
[root@localhost csdk]# tcpdump -i ens33 port 25625 -w /home/dump1.pcap
tcpdump: listening on ens33, link-type EN10MB (Ethernet), capture size 65535
bytes
^C297 packets captured
297 packets received by filter
0 packets dropped by kernel
```

如果显示的为“0 packets received by filter”，则说明tcpdump设置错误。请重新设置。
抓取成功后的网络包数据文件为“/home/dump1.pcap”

4、使用wireshark打开步骤3生成的pcap文件，本示例中的文件为dump1.pcap



如果测试环境没有其他会话的干扰，且捕捉的内容为完整的客户端登录。则第1个有长度的网络包即为ssl的第一个包即Client Hello包。

在上例中，第七个包即为SSL/TLS协议中的“**Client hello**”包
通过该包，能够看到本次ssl通讯使用的版本为“**TLSv1.2**”。

12.1.11 附录

附件A： 示例证书

该示例证书用于演示GBase 8s产品如何使用SSL/TLS通讯。

注： 以下示例证书的私钥保护口令为： 11111111

certs_rsa.zip: RSA算法证书（国际算法）

certs_sm2.zip: 国密算法证书（仅最新版8s支持）



certs_rsa.zip



certs_sm2.zip

附件B： 生成RSA证书

该脚本用于生成RSA算法（使用SHA256算法签名）的证书。

包含CA、server端、用户证书。

本示例生成的证书为x509格式，证书后缀名为“crt”，私钥文件后缀名为“key”，证书请求文件的后缀名为“csr”，jdk/jre使用的证书后缀名为“jks”。当证书文件后缀为pfx时，该证书为PKCS12格式。

以此脚本生成证书的私钥保护口令为“11111111”。



gen_rsa_cert.sh

附件B： 生成国密算法证书

以下脚本内容支持生成SM2算法（使用SM3算法签名）的证书。

包含CA、server端、用户证书。

本示例生成的sm2证书为x509格式，证书后缀名为“crt”，私钥文件后缀名为“key”，证书请求文件的后缀名为“csr”。当证书文件后缀为pfx时，该证书为PKCS12格式。

以此脚本生成证书的私钥保护口令为“11111111”。



gen_sm2_cert.sh

附录D： 证书文件格式说明

从分类标准上分，SSL 证书格式主要有

- 公钥证书格式标准 X.509 中定义的 PEM 和 DER
- 公钥加密标准 PKCS 中定义的 PKCS#7 和 PKCS#12

- Java 环境专用的 JKS

从文件格式上分，SSL 证书格式主要有：

- Base64 (ASCII) 编码的文本格式。这种证书文件是可以通过文本编辑器打开，甚至可以编辑，常见有 PEM 证书格式，扩展名包括 PEM、CRT 和 KEY。
- Binary 二进制文件。常见有 DER 证书格式，扩展名包括 DER 和 CER。

Linux 系统使用 CRT，Windows 系统使用 CER。

名词	含义
X.509	一种通用的证书格式，包含证书持有人的公钥，加加密算法等信息
pkcs1 ~pkcs12	公钥加密（非对称加密）的一种标准(Public Key Cryptography Standards)，一般存储为 .pN，.p12 是包含证书和密的封装格式
*.der	证书的二进制存储格式（不常用）
*.pem	证书或密钥的 Base64 文本存储格式，可以单独存放证书或密钥，也可以同时存放证书或密钥
*.key	单独存放的 pem 格式的密钥，一般保存为 *.key
*.cer *.crt	两个指的都是证书，Linux 下叫 crt，Windows 下叫 cer；存储格式可以是 pem，也可以是 der
*.csr	证书签名请求（Certificate signing request），包含证书持有人的信息，如：国家，邮件，域名等信息
*.pfx	微软 IIS 的实现
*.jks	Java 的 keytool 实现的证书格式
*.truststore	java的证书仓库，存放一个或多个CA 这里的truststore就像你自己电脑的证书管理器一样，如果你打开Chrome的设置，找到HTTP SSL，就可以看到里面有很多CA，truststore就是干这个活儿的，它也里面也是存一个或多个CA让Tomcat或Java程序来调用。

*.keystore	Java的密钥仓库，存放一个或多个key 而keystore就是用来存密钥文件key的，可以存放多个。
------------	--

总结：

首先证书链必须要有一个CA（certificate authority），客户端和服务端的TLS/SSL连接必须使用相同的CA才能完成认证。

加密通讯时的消息流向如下：

消息-->[公钥]-->加密后的信息-->[私钥]-->消息

公钥只能用做数据加密。公钥加密的数据，只能用对应的私钥才能解密。这是非对称加密的核心概念。

- ASN.1是一套完整的数据结构与数据存储格式描述
- BER/DER是ASN.1的二进制编码方式
- PEM是DER编码再进行Base64编码后的x文本格式

这三种是证书文件的编码格式，和文件名后缀、内容无关。

CSR是证书签名请求，包含有证书持有人的信息，例如：国家、city、邮编等等。CA用自己的私钥文件签名之后生成的文件就是完整的证书了即 .cer文件（或linux下的.crt文件）。

所以，生成证书的流程就是：

- 1、 首先需要提供CA的证书：生成（用自己提供的私钥做自签）或直接使用第三方提供的CA证书（crt文件）
- 2、 生成该证书对应的私钥：指定算法等信息，生成对应的key文件；
- 3、 生成证书签名的请求：主要是填写国家、城市等证书的信息，以及上一步获得的私钥key文件，生成csr文件。
- 4、 使用CA私钥对该证书请求进行签名，获得crt文件。
- 5、 将crt转换为pkcs#12格式的证书文件pfx文件。
- 6、 调用keytool（JDK自带）或Genkey（附件H），将pfx文件证书转换为jks格式证书。

附录E： 如何确认服务端tls版本

可以使用openssl工具（参考章节一），获取服务端tls/ssl的版本。

例如，验证部署在192.168.0.101（端口为25625）的ids端服务是否支持tls1.3，可用如下方法：

```
./bin/openssl s_client -connect 192.168.0.101:25625 -tls1_3
```

说明：此openssl为章节一中编译的3.2版本的openssl。

执行后返回结果为：

```
Connecting to 192.168.0.101
CONNECTED(00000004)
4047A987087F0000:error:0A000410:SSL routines:(unknown function):ssl/tls
alert handshake failure:ssl/record/rec_layer_s3.c:909:SSL alert number 40
---
no peer certificate available
---
No client certificate CA names sent
---
SSL handshake has read 7 bytes and written 229 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```


同时，再去验证该ids服务是否支持tls1.2，则用如下命令：

```
./bin/openssl s_client -connect 192.168.0.101:25625 -tls1_2
```

则验证的结果如下：

```
Connecting to 192.168.0.101
CONNECTED(00000004)
Can't use SSL_get_servername
depth=1 C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
verify error:num=19:self-signed certificate in certificate chain
verify return:1
depth=0 C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=8s_server
verify error:num=68:CA signature digest algorithm too weak
verify return:1
depth=1 C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
verify return:1
depth=0 C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=8s_server
verify return:1
---
Certificate chain
 0 s:C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=8s_server
   i:C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA1
   v:NotBefore: Nov  9 02:57:29 2021 GMT; NotAfter: Nov  7 02:57:29 2031
GMT
 1 s:C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
   i:C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
```

```
a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA1

v:NotBefore: Nov  9 02:57:28 2021 GMT; NotAfter: Nov  7 02:57:28 2031
GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDSjCCApqgAwIBAgIJAMuOVkDsS6MPMA0GCSqGSIb3DQEBBQUAMFkxCz
AJBgNV
BAYTAkNOMQswCQYDVQQIDAJUSjELMAkGA1UEBwwCVEoxDjAMBgNVBAo
MBUdCYXNI
MQ4wDAYDVQQQLDAVHQmFzZTEQMA4GA1UEAwwHR0Jhc2U4czAeFw0yMTE
xMDkwMjU3
MjlaFw0zMTEwMDcwMjU3MjlaMFsxZzAkJBgNVBAYTAkNOMQswCQYDVQQID
AJUSjEL
MAkGA1UEBwwCVEoxDjAMBgNVBAoMBUdCYXNIMQ4wDAYDVQQQLDAVHQ
mFzZTESMBAG
A1UEAwwJOHNfc2VydmVyMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgK
CAQEA
vLmZC6rHbGotNM8aYN5cdjjxEgh5rhziDHFSDh/P1L1pNbN0m0lbhiwpmwD5
w8h3
YoYBZwJvkoBiFEIA+FrLuMghyaVlf764wmOitQfuhbsYwxh/kS/aP2AOh3sCqZz
N
g6pVBKYkYBEgXPOoC2uKRh+M3qLJPOMzD9dSNhPJFZBRV8NtsKqsLCp4XnJj
SgVB
pDM/XMk6yKqsK96NKFwcdvtXiJSu/S+YvLnN47+hz7gux/g9D9/nnaq3blAiDH
```

```
p4
PvJFDmjvsbXeRRm54+qY/9wKBkfssxibZrFgB5P8qwh7tJBTWHAifXzrGnm5Vd
UK
08utpJLofhAuKy7adRBjDwIDAQABo3sweTAJBgNVHRMEAjAAMCwGCWCGSA
GG+EIB
DQQfFh1PcGVuU1NMIEdlbmVyYXRIZCBZCDBZJ0aWZpY2F0ZTAdBgNVHQ4EFg
QUy69h
sIS7WQ11UvAIFph9aLYx130wHwYDVR0jBBgwFoAUxihGumIPsCJEgL+XFjY137
wo
5bEwDQYJKoZIhvcNAQEFBQADggEBADAs2td8xX3JMCBuOS6xltgbiEXbwD0T
D8oL
dyQuAfrKi85NCtzWh1KSLZpp5MUY+JsGRNc0NbJhlQCrX2iq19q1CkISNQECS
Ojp
CmMw262V758J1USjKkjr0JqFZVPDAZW+n0zmTvYPp0MO7pSXkz9IsjEbKRBoy
dH1
FoF+JEdz0bsu4tSgzAXRFFymddDf2NJRxCp5MnJQ+TwSsRpQp7eUA/wrH1jkh
K7w
1r4EMAJvHimmYCTLe3CEFOYijvOZRdSnWhByqHEGjHfQR9AkGOJz8Hvj8ygdT
e8M
g7RRU94sgyxzUupKYMkzFXJLICibqhVkyLe13jM3Bt2qGSPMsnI=
-----END CERTIFICATE-----
subject=C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=8s_server
issuer=C=CN, ST=TJ, L=TJ, O=GBase, OU=GBase, CN=GBase8s
---
No client certificate CA names sent
```

```
---  
  
SSL handshake has read 2166 bytes and written 512 bytes  
Verification error: CA signature digest algorithm too weak  
  
---  
  
New, TLSv1.2, Cipher is AES256-GCM-SHA384  
Server public key is 2048 bit  
Secure Renegotiation IS supported  
Compression: NONE  
Expansion: NONE  
No ALPN negotiated  
SSL-Session:  
    Protocol   : TLSv1.2  
    Cipher     : AES256-GCM-SHA384  
    Session-ID:  
5A101E7A2DC1B5DA90725A0C71EB865A3B3CC399FFD8A6E83E3BD544B807  
9913  
    Session-ID-ctx:  
    Master-Key:  
599BAB714A15EE09391FECDD04584EB55EC778B8075745D55E3390AA800D83  
786B77A0ABBB5FEBED85CF42DA981B093BB  
    PSK identity: None  
    PSK identity hint: None  
    SRP username: None  
    TLS session ticket lifetime hint: 300 (seconds)  
    TLS session ticket:
```

```
0000 - 28 38 87 cb ef d6 e6 ff-a7 df b2 1e ce 90 09 87 (8.....
0010 - 50 32 93 51 8f 3c 13 ba-22 b5 37 1f f9 be 54 5b P2.Q.<.."7...T[
0020 - d1 96 65 5b 11 02 fb 41-88 40 05 4b cb a5 82 06 ..e[...A.@.K...
0030 - e2 6f 96 79 de c8 b7 55-cc b2 c2 63 d5 e2 42 63 .o.y...U...c..Bc
0040 - ae c5 82 40 18 a3 ae 9d-4d aa ab 4a 35 35 86 1a ...@....M..J55..
0050 - ab 66 2d ac 3c 0d ef d8-db 67 2c 3f 9d ad 43 b9 .f-.<....g,?...C.
0060 - a8 39 2c c4 02 ad 07 93-b7 65 57 8a 35 97 c3 5a .9,.....eW.5..Z
0070 - 90 f2 ac 46 e3 1e 1d b3-4a 47 10 b9 f5 4f 8f c8 ...F....JG...O..
0080 - 5a 55 79 3b f2 ab c9 fe-42 be cd 0a 47 30 77 a1 ZUy;....B...G0w.
0090 - 2e f7 69 0e e9 f6 e7 5f-57 42 7d f7 fd 4c 2a 2c ..i...._WB}..L*,
```

Start Time: 1720170849

Timeout : 7200 (sec)

Verify return code: 68 (CA signature digest algorithm too weak)

Extended master secret: no

400769478D7F0000:error:0A000126:SSL routines::unexpected eof while
reading:ssl/record/rec_layer_s3.c:694:

由于未配置对应的证书，openssl在tls1.2时保持了各种验证错误，但显然，位于192.168.0.101服务器上的ids，无法支持tls1.3而能够支持tls 1.2。

附录F： \${GBASEDBTSQLHOSTS}.ext的规格

无论客户端，连接管理器，还有数据库服务端都存在配置文件。该配置文件都是通过\${GBASEDBTSQLHOSTS}.ext指定的。

该文件所属的组是gbasedbt，所属的用户是gbasedbt，权限是664。

该文件的内容如下：

```
[config]

GBS_TYPE=gbasessl    #GBS的类型。字符串，最大长度31。目前仅支持gbasessl
                      ( 也就是使用openssl完成传输加密 )

[ids_name]    #ids服务名或连接管理器名，需要和sqlhosts中的名字保持一致

server=ids_name_server_certs  #当前程序是ids_instance_name 的server端，对
应的安全配置的section名。

client=ids_name_client_certs  //当前程序是ids_instance_name的client端，对应
的安全配置的section名

[ids_name_server_certs]

TLSCACertificateFile=<full_path> //CA证书。字符串,最大长度511。

TLSCertificateFile=<full_path>  //服务器证书。字符串，最大长度511。

TLSCertificateKeyFile=<full_path> //服务器证书私钥。字符串，最大长度511。

TLSCertificateKeyFilePasswd=<password> //服务器证书私钥保护密码。字符串，
最大长度511，要求用密文保存。

TLSVerifyCert={verify | try | never} //是否验证对方证书

    //verify: 要求对方必须发送证书,并且验证证书.

    //try: 要求对方发送证书.如果对方不发送,则不验证; 如果对方发送,则验证证书.

    //never: 不要求对方发送证书.

[ids_name_client_certs]

TLSCACertificateFile=<full_path> //CA证书。字符串,最大长度511。

TLSCertificateFile=<full_path>  //客户端证书。字符串，最大长度511。
```

```
TLSCertificateKeyFile= <full_path> //客户端证书私钥。字符串，最大长度511。  
TLSCertificateKeyFilePasswd= <password> //客户端证书私钥保护密码。字符串，  
最大长度511，要求用密文保存。  
TLSVerifyCert={verify | try | never} //是否验证对方证书  
  
//verify: 要求对方必须发送证书,并且验证证书。  
  
//try: 要求对方发送证书.如果对方不发送,则不验证; 如果对方发送,则验证证书。  
  
//never: 不要求对方发送证书。
```

TLSCertificateKeyFilePasswd要求使用密文保存。需要使用gbspwdenc加密后保存在该配置文件中。

12.2 附录G： gbspwdenc工具说明

该工具的功能是将输入的字符串使用AES算法进行加密，再使用BASE64进行编码，最后将结果输出到屏幕上。

该工具部署在ids和csdk的bin目录中。其用户组以及权限，在ids中和oninit一致；在csdk中和 oncmsm一致。

使用方式，加密口令明文使用单引号包围：

```
gbspwdenc '加密口令明文'
```

输出：加密口令密文

加解密算法：

加密：（gbspwdenc使用）

- 1）使用AES算法将密码明文加密成密文，密钥为"7tfc4rgn0okm7il!"，密码在数据段中以异或后的值存储，使用时再异或回来。
- 2）使用base64算法将密文转换为可见字符。

解密：（gbasesecurity模块在解析配置文件时使用）

1) 使用base64解码，再用AES算法（密钥和加密相同）解码。

可以支持的字符集合：

1234567890

abcdefghijklmnopqrstuvwxyz

ABCDEFGHIJKLMNOPQRSTUVWXYZ

`~!@#\$%^&*()-_=[{}];:~" \|, < . > / ?

空格

注：常见可见字符都可以直接支持，除了单引号’。如果要使用单引号，则需要使用双引号包围单引号。

比如加密口令明文是：123’ 456

则需要输入：

gbspwdenc ‘123’ “ ’ ” ’ 456’

加密口令明文长度限制最长为128。

12.3 附件H： java key store生成

该附件用于将pkcs#12格式（pfx文件）的证书转换为java使用jks格式。



GenJKS.java



GenJKS.class

12.4 参考资料

基于BouncyCastle的证书格式转换demo编写

https://www.cnblogs.com/fzlzc/p/14600216.html#_label5

Interface X509ObjectIdentifiers

<https://downloads.bouncycastle.org/fips-java/docs/bc-fips-1.0.2.4-javadoc/>

Java密钥库及keytool使用详解

<https://www.cnblogs.com/lihw-study/p/15591831.html#166--providername>

国密SSL协议之Java编程

<https://segmentfault.com/a/1190000023890321>

使用 Tongsuo 签发 SM2 证书

<https://www.yuque.com/tsdoc/ts/pb5vqr>

GBASE[®]

南大通用数据技术股份有限公司
General Data Technology Co., Ltd.



微信二维码

■ ■ 技术支持热线：400-013-9696

